

SymbolFit: Machine can automatically discover parametric functions to model HEP data

Ho Fung Tsoi [[UPenn](#)], Dylan Rankin [[UPenn](#)], Cecile Caillol [[CERN](#)], Miles Cranmer [[Cambridge](#)],
Sridhara Dasu [[UW-Madison](#)], Javier Duarte [[UCSD](#)], Philip Harris [[MIT](#)], Elliot Lipeles [[UPenn](#)]

ICHEP, 30 July 2026

Natal, Brazil

Paper: [2411.09851](#) (API) / [2607.19750](#) (dijet function rediscovery)

Code: [github:hftsoi/symbolfit](#)

Docs: [hftsoi.github.io/symbolfit](#)

SymbolFit: Automatic Parametric Modeling with Symbolic Regression

Ho Fung Tsoi¹, Dylan Rankin¹, Cecile Caillol², Miles Cranmer³, Sridhara Dasu⁴,
Javier Duarte⁵, Philip Harris^{6,7}, Elliot Lipeles¹, Vladimir Loncar^{6,8}

¹University of Pennsylvania, USA

²European Organization for Nuclear Research (CERN), Switzerland

³University of Cambridge, UK

⁴University of Wisconsin-Madison, USA

⁵University of California San Diego, USA

⁶Massachusetts Institute of Technology, USA

⁷Institute for Artificial Intelligence and Fundamental Interactions, USA

⁸Institute of Physics Belgrade, Serbia

E-mail: ho.fung.tsoi@cern.ch

Abstract. We introduce SymbolFit[†], a framework that automates parametric modeling by using symbolic regression to perform a machine-search for functions that fit the data while simultaneously providing uncertainty estimates in a single run. Traditionally, constructing a parametric model to accurately describe binned data has been a manual and iterative process, requiring an adequate functional form to be determined before the fit can be performed. The main challenge arises when the appropriate functional forms cannot be derived from first principles, especially when there is no underlying true closed-form function for the distribution. In this work, we develop a framework that automates and streamlines the process by utilizing symbolic regression, a machine learning technique that explores a vast space of candidate functions without requiring a predefined functional form because the functional form itself is treated as a trainable parameter, making the process far more efficient and effortless than traditional regression methods. We demonstrate the framework in high-energy physics experiments at the CERN Large Hadron Collider (LHC) using five real proton-proton collision datasets from new physics searches, including background modeling in resonance searches for high-mass dijet, trijet, paired-dijet, diphoton, and dimuon events. We show that our framework can flexibly and efficiently generate a wide range of candidate functions that fit a nontrivial distribution well using a simple fit configuration that varies only by random seed, and that the same fit configuration, which defines a vast function space, can also be applied to distributions of different shapes, whereas achieving a comparable result with traditional methods would have required extensive manual effort.

Machine Can Automatically Discover Parametric Functions to Model HEP Data

Ho Fung Tsoi,^{a,*} Dylan Rankin,^a Cecile Caillol,^b Miles Cranmer,^c Sridhara Dasu,^d Javier Duarte,^e Philip Harris^{f,g} and Elliot Lipeles^a

^a*University of Pennsylvania, USA*

^b*European Organization for Nuclear Research (CERN), Switzerland*

^c*University of Cambridge, UK*

^d*University of Wisconsin-Madison, USA*

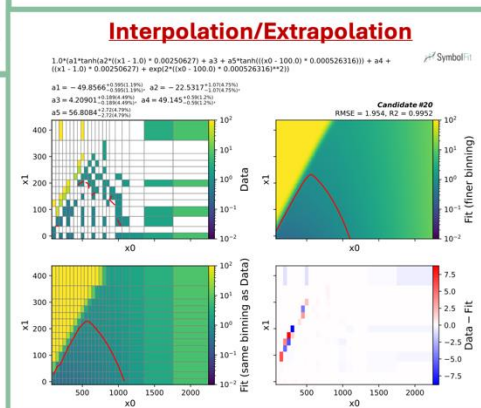
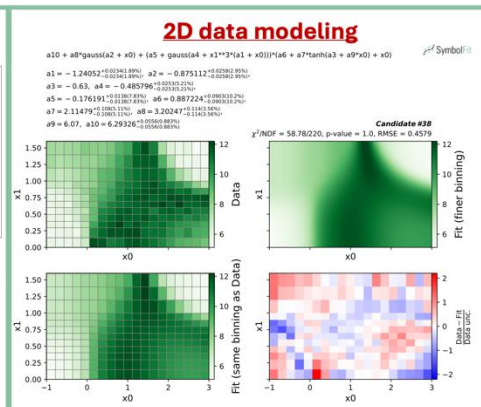
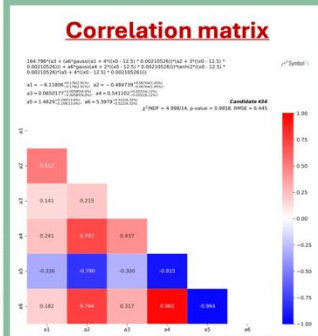
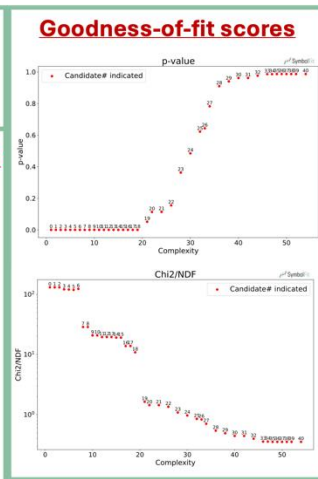
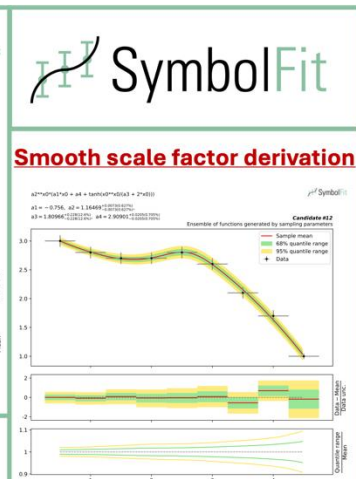
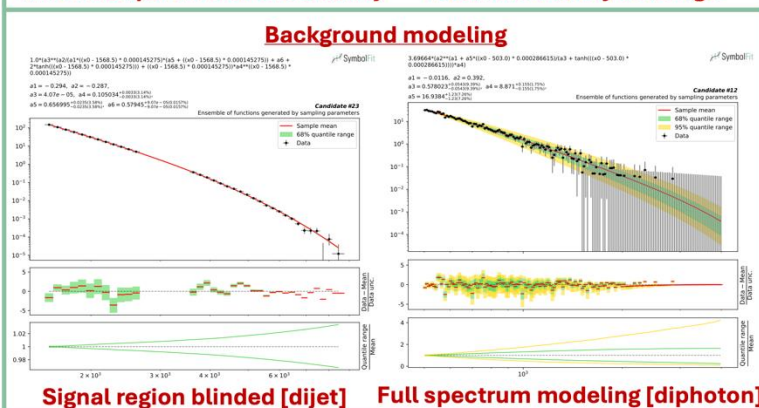
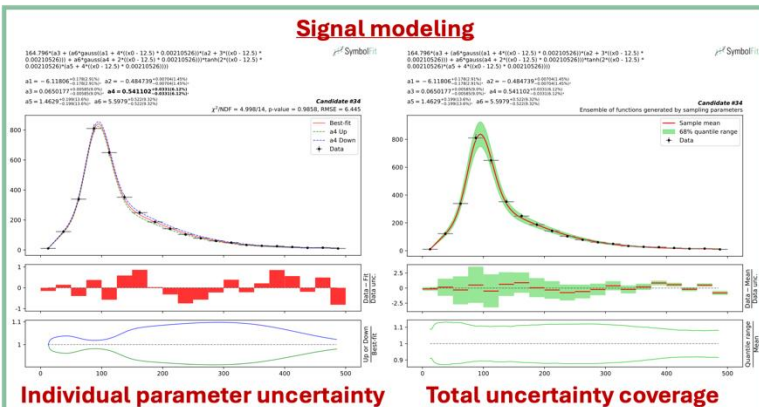
^e*University of California San Diego, USA*

^f*Massachusetts Institute of Technology, USA*

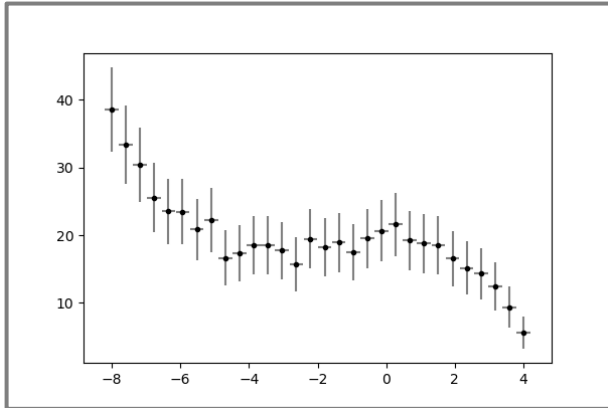
^g*Institute for Artificial Intelligence and Fundamental Interactions, USA*

E-mail: ho.fung.tsoi@cern.ch

In HEP data analyses, finding an adequate function to model binned data has largely relied on a manual process: guess a functional form by intuition, fit, examine, then repeat until successful. We show that this iterative process can be automated by a machine using symbolic regression, which performs a data-driven search over function space without requiring prior knowledge of what an adequate function should look like. We present the SymbolFit package, which pairs symbolic regression with uncertainty modeling to target HEP analysis use cases, and demonstrate it on the CMS and ATLAS Run 2 dijet spectra: 560 independent seeded runs across seven simple fit configurations generated over 1000 functions fitting the spectra with $\chi^2/\text{NDF} \approx 1$, and 111 of the runs rediscovered the very dijet and UA2 functions used in published dijet searches.



Problem statement



Given a distribution, how to derive a smooth function to model it?

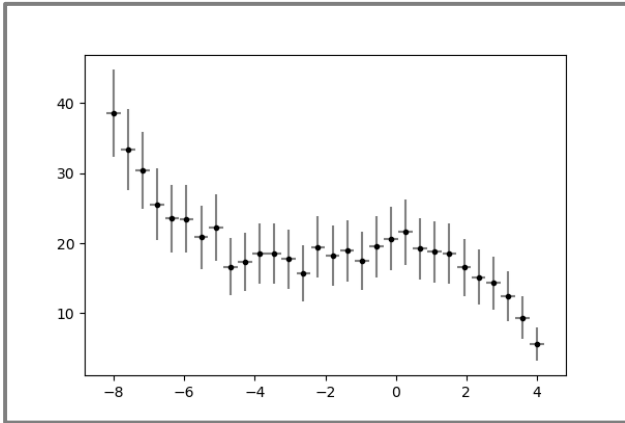
Applications in HEP analysis

- Sig/bkg modeling in resonance analyses (e.g., $h \rightarrow \gamma\gamma$, dijet,...)
- Derive smooth data/MC scale factor from binned measurements (e.g., trigger SFs)
- Derive smooth transfer factor for bkg modeling (e.g., ABCD method)

The traditional way is: (1) guess a function by intuition $\rightarrow$ (2) write parameterization by hand $\rightarrow$ (3) fit and examine; then repeat with another guess until successful.

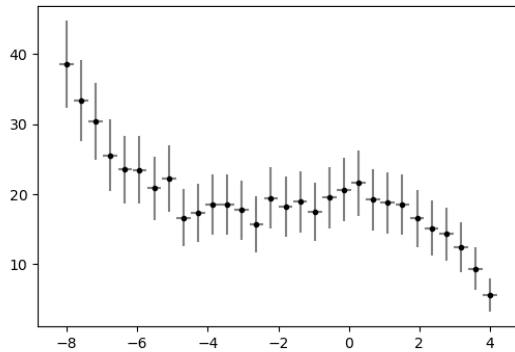
This has been the standard practice in HEP for decades, even as of today, which is painful, can we overcome this?

The pain point of the traditional way



The pain point of the traditional way

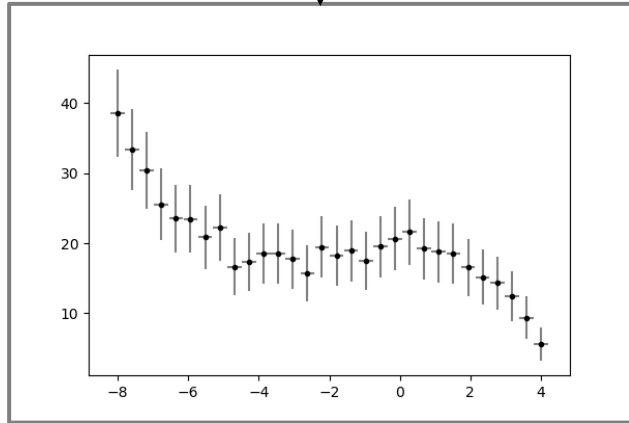
Understand the distribution
shape



The pain point of the traditional way

Understand the distribution shape

Guess a function form by intuition
(polynomial, exponential,...)

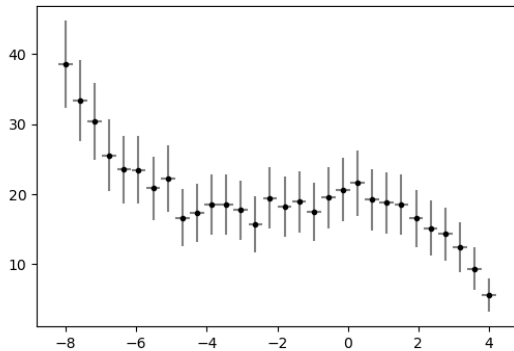


The pain point of the traditional way

Understand the distribution shape

Guess a function form by intuition
(polynomial, exponential,...)

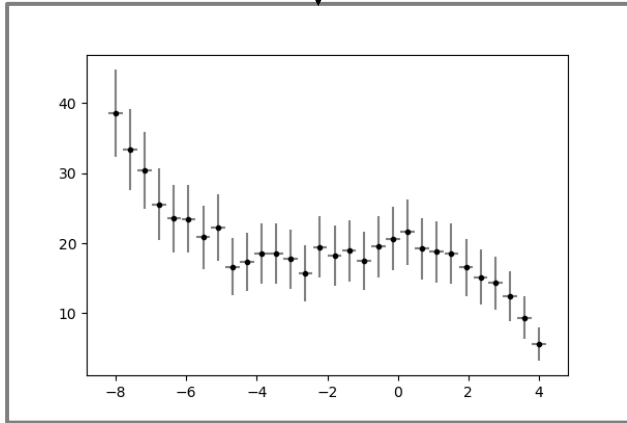
Fit to data and examine



The pain point of the traditional way

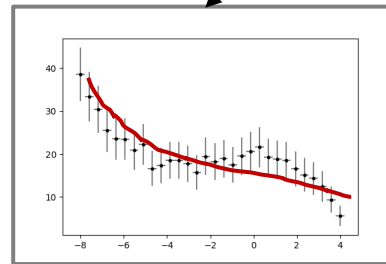
Understand the distribution shape

Guess a function form by intuition (polynomial, exponential,...)

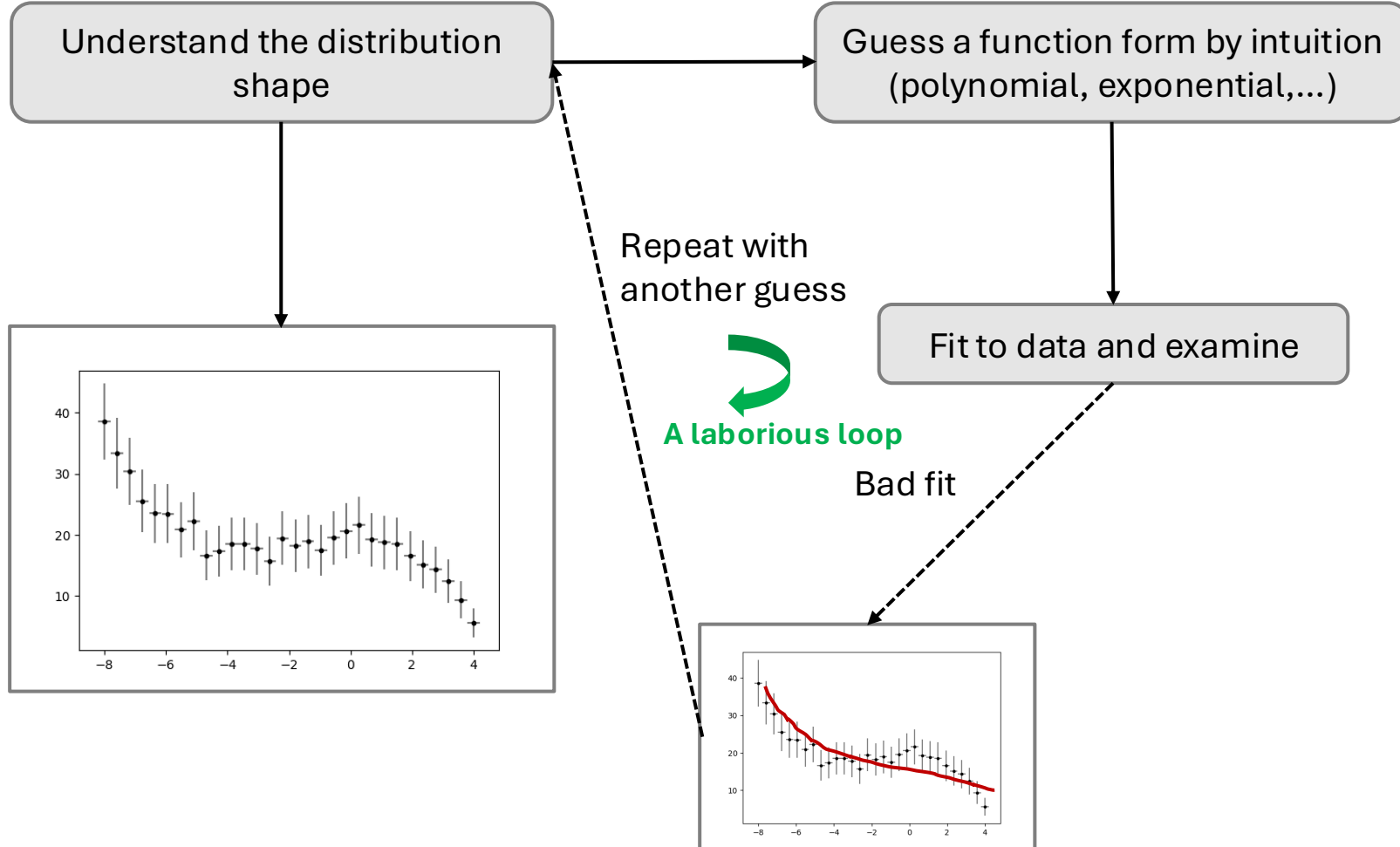


Fit to data and examine

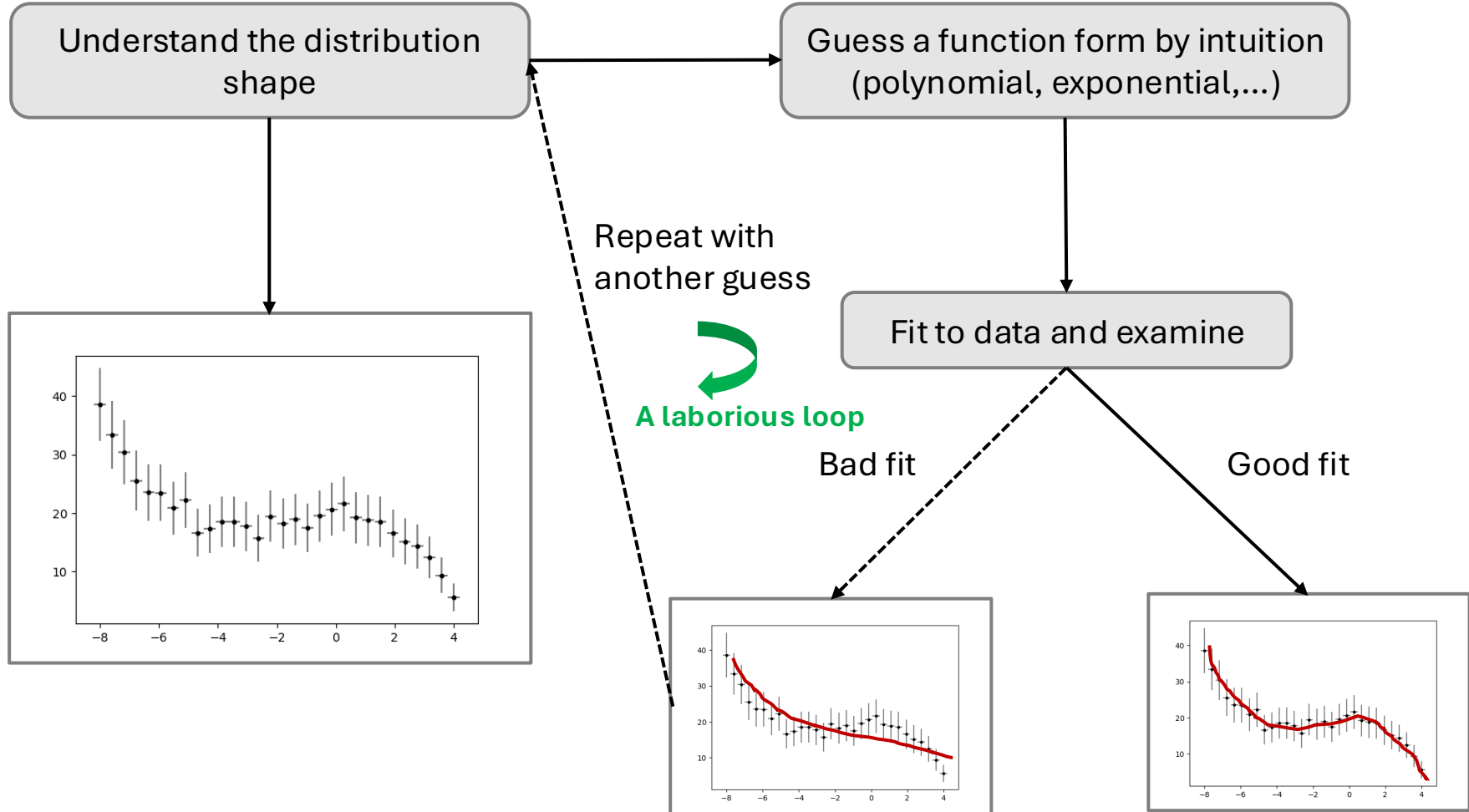
Bad fit



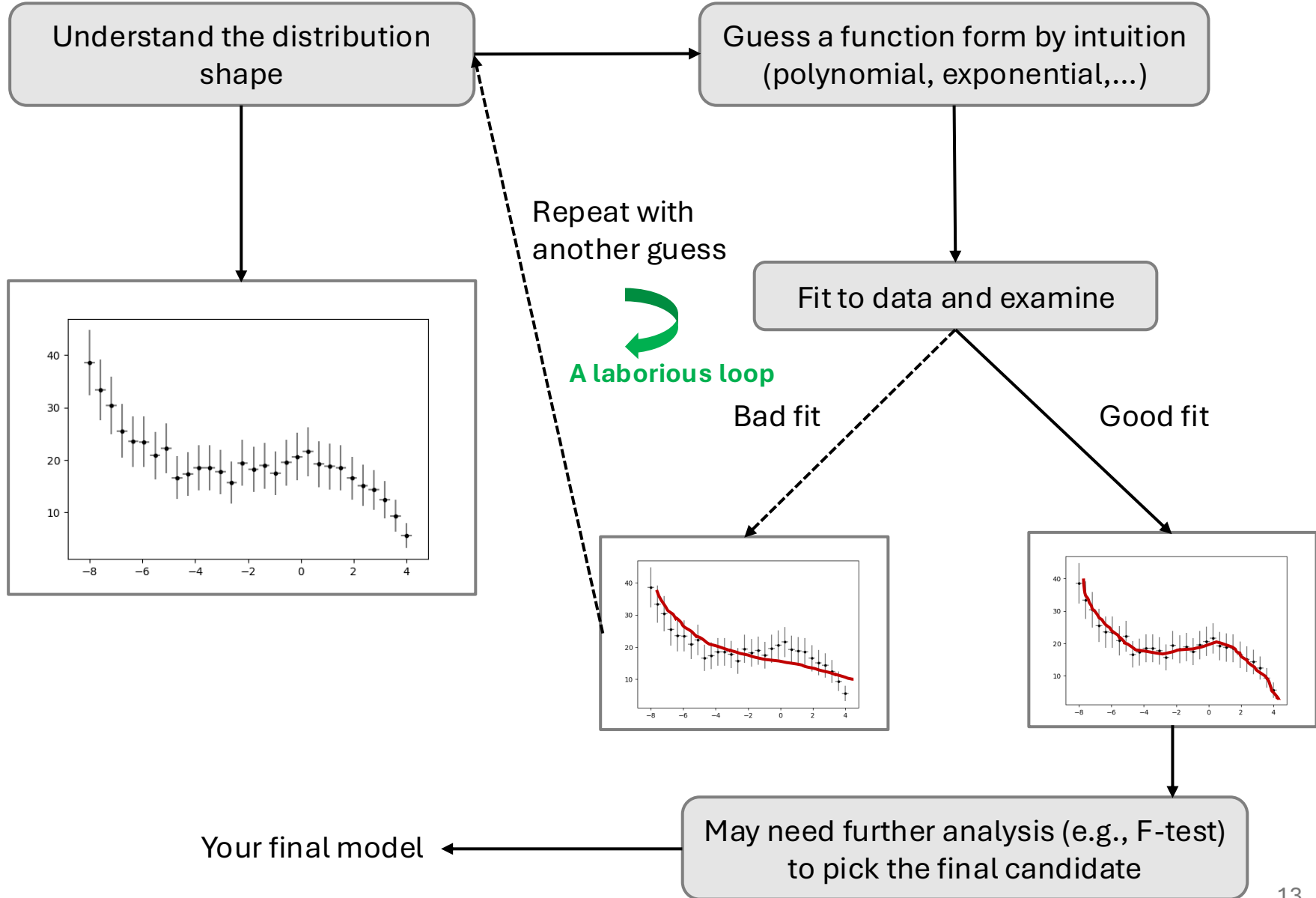
The pain point of the traditional way



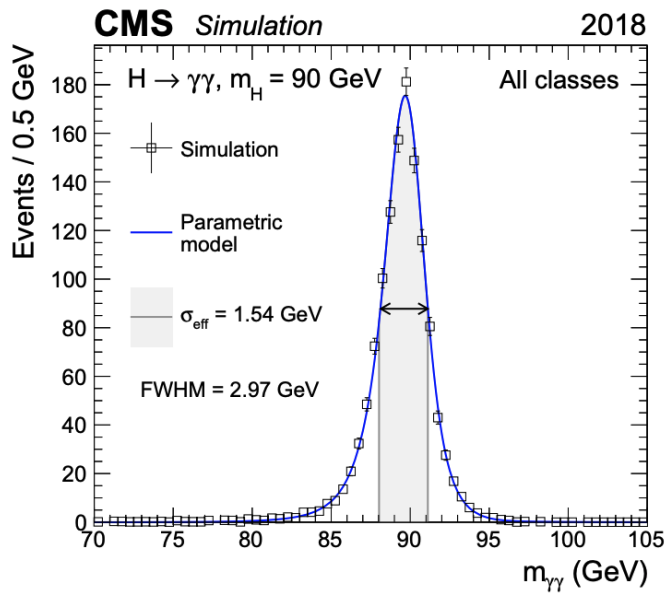
The pain point of the traditional way



The pain point of the traditional way



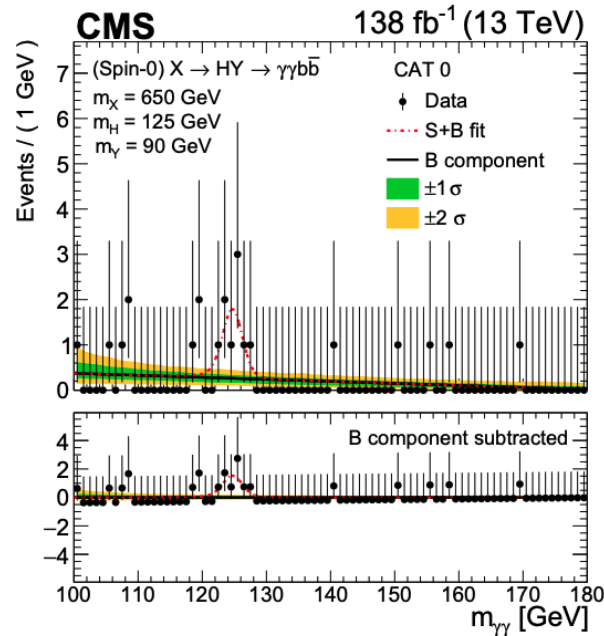
The pain point of the traditional way



[2405.18149](#)

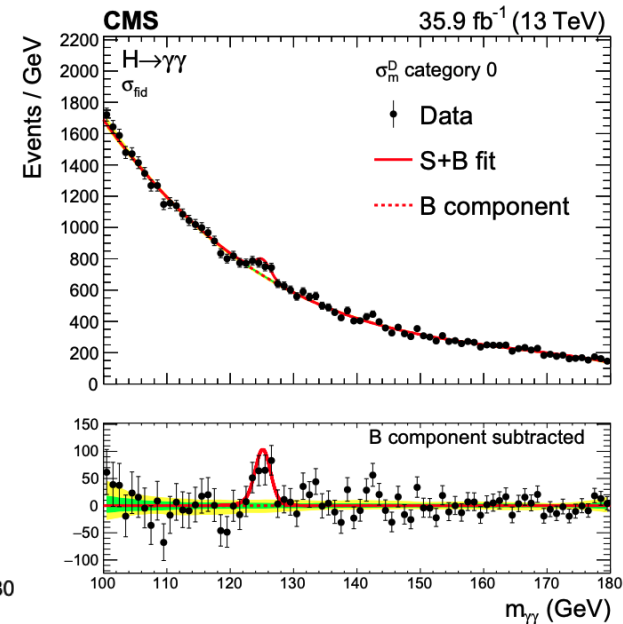
“...is modeled empirically with a sum of between two and four Gaussian functions...”

“...the number of Gaussian functions is determined using an F-test...”



[2310.01643](#)

“...The $m_{\gamma\gamma}$ distribution is parametrized using the sum of up to five Gaussian functions...”

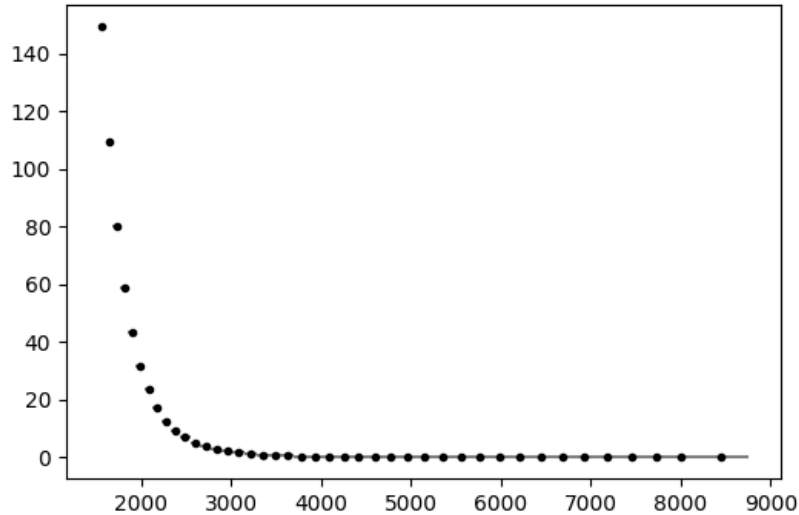


[1807.03825](#)

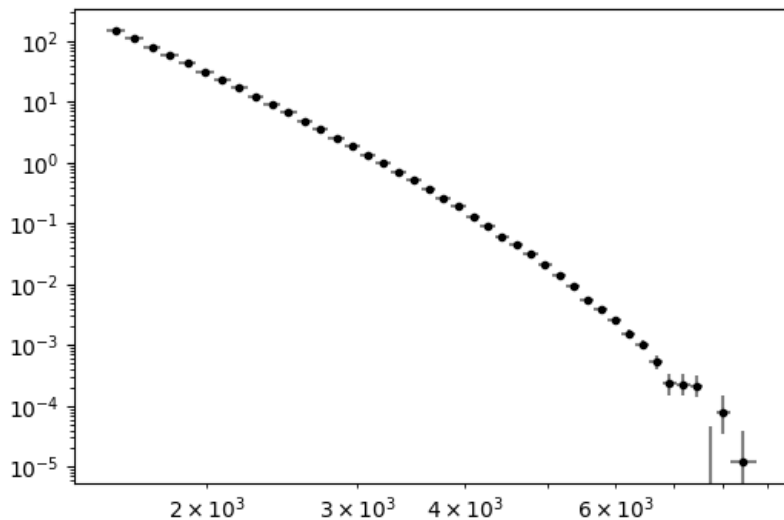
“...The model is built as a fit to a sum of up to five Gaussian distributions of the simulated invariant mass shape...”

- Peaked signals are easy, the hard part is background

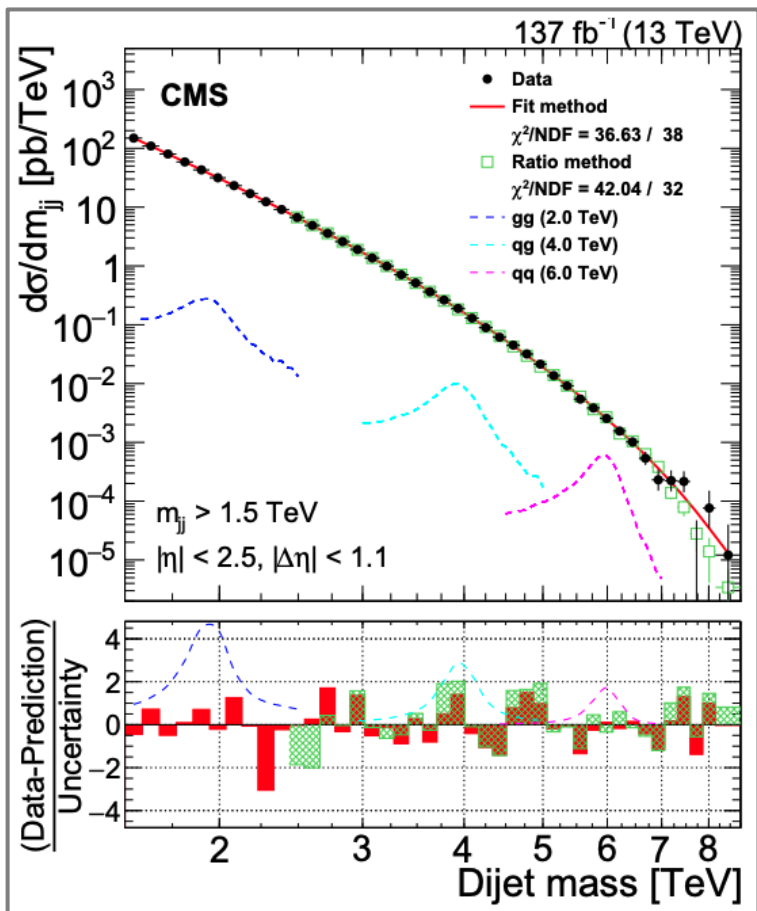
The pain point of the traditional way



- E.g., CMS dijet spectrum [[1911.03947](#)]
- Looks steeply falling, but simple exponentials or powers won't work



The pain point of the traditional way



CMS Run 2 dijet resonance search [1911.03947](#)

5 Background prediction methods

In the fit method, utilized here and in previous dijet resonance searches [17, 19–32, 50], the main background in the SR coming from QCD is parametrized with an empirical function of the form

$$\frac{d\sigma}{dm_{jj}} = \frac{P_0(1-x)^{P_1}}{x^{P_2+P_3 \ln(x)}}, \quad (1)$$

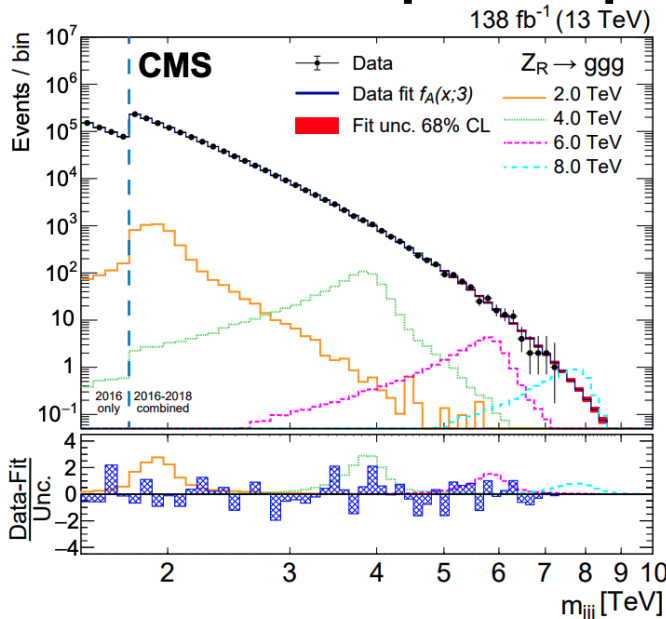
where $x = m_{jj}/\sqrt{s}$, and P_0 , P_1 , P_2 , and P_3 are four free parameters. The search for resonances

- CMS/ATLAS hand-crafted several “dijet functions” empirically
- Each one is rigid and new one is invented for new analysis

[2206.09997](#)

larger event samples. These are the modified dijet function (ModDijet-5p) given by $d\sigma/dm = p_0(1-x^{1/3})^{p_1}/(x^{p_2+p_3 \log x+p_4 \log^2 x})$, the dijet function (Dijet-5p) given by $d\sigma/dm = p_0(1-x)^{p_1}/(x^{p_2+p_3 \log x+p_4 \log^2 x})$, and the power-law times exponential function (PowExp-5p) given by $d\sigma/dm = p_0 e^{-p_1 x - p_2 x^2 - p_3 x^3}/x^{p_4}$, where $x = m/\sqrt{s}$, with m being the four-jet mass.

The pain point of the traditional way



CMS Run 2 high-mass trijet search [2310.14023](#)

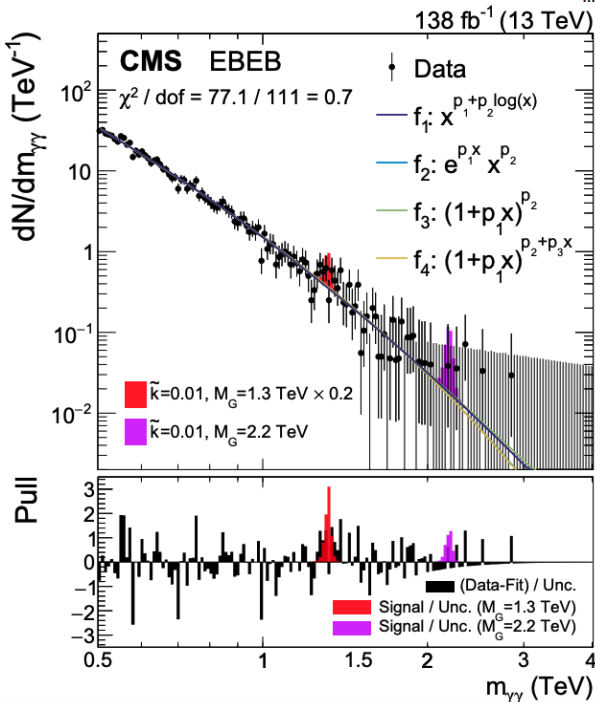
The background, dominantly arising from the QCD multijet process, is modeled by fitting the m_{III} distribution with smoothly falling, empirical functions that have been used in previous searches [19, 20]. Three families of functions are considered:

$$f_A(x; N) = p_0 \frac{(1-x)^{p_1}}{x^{\sum_{i=2}^N p_i \log^{i-2}(x)}},$$

$$f_B(x; N) = p_0 \frac{e^{-p_1 x}}{x^{\sum_{i=2}^N p_i \log^{i-2}(x)}},$$

$$f_C(x; N) = p_0 x^{\sum_{i=1}^N p_i \log^{i-1}(x)},$$
(1)

where $x = m_{III} / \sqrt{s}$, p_i ($i = 0, 1, \dots, N$) are free parameters, and N is the order of the fit function, which is determined using a Fisher F-test [48]. It is found that the third order yields the



CMS Run 2 high-mass diphoton search [2405.09320](#)

5.1 Resonant search background

For the resonant signal search, parametrized functions are fit to the $m_{\gamma\gamma}$ spectrum. Localized excesses in the form of a peak are distinguishable from the smoothly-falling background. Four functions of the dimensionless variable $x = m_{\gamma\gamma} / \sqrt{s}$ are given equal consideration in the statistical analysis:

$$f_1(x) = p_0 x^{p_1+p_2 \log(x)},$$

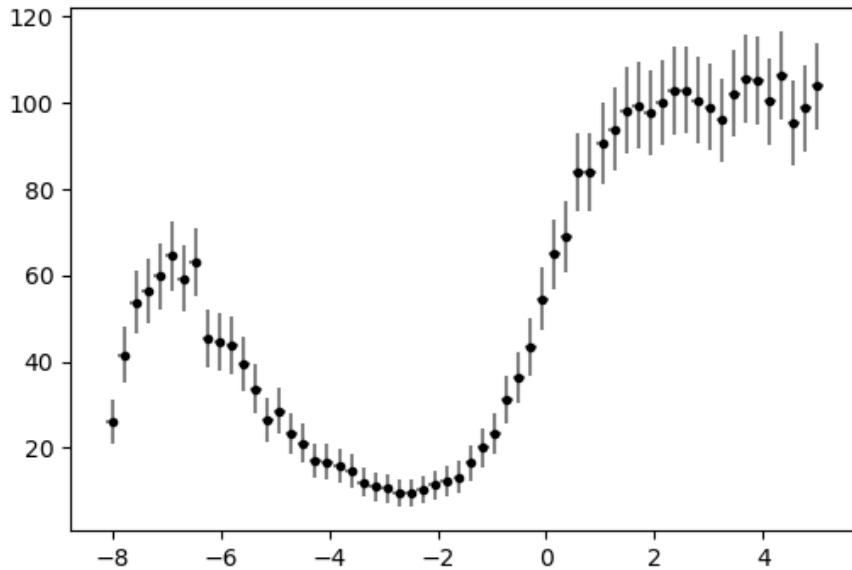
$$f_2(x) = p_0 e^{p_1 x} x^{p_2},$$

$$f_3(x) = p_0 (1 + x p_1)^{p_2},$$

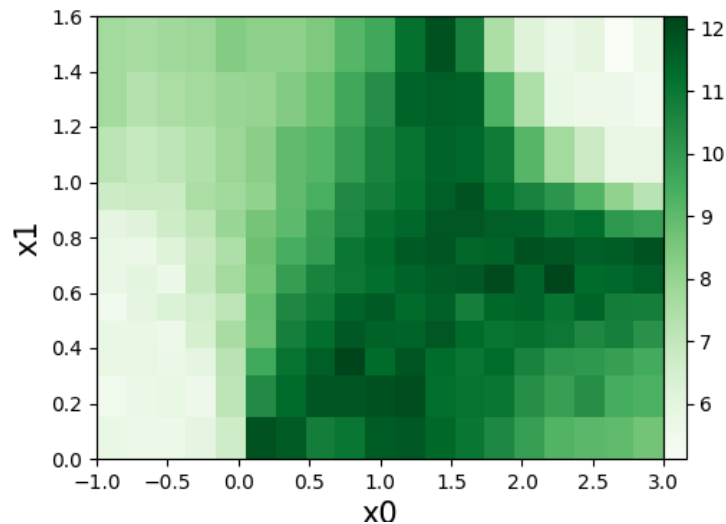
$$f_4(x) = p_0 (1 + x p_1)^{p_2+p_3 x},$$
(4)

where p_i are unconstrained nuisance parameters in the hypothesis test. During the likelihood

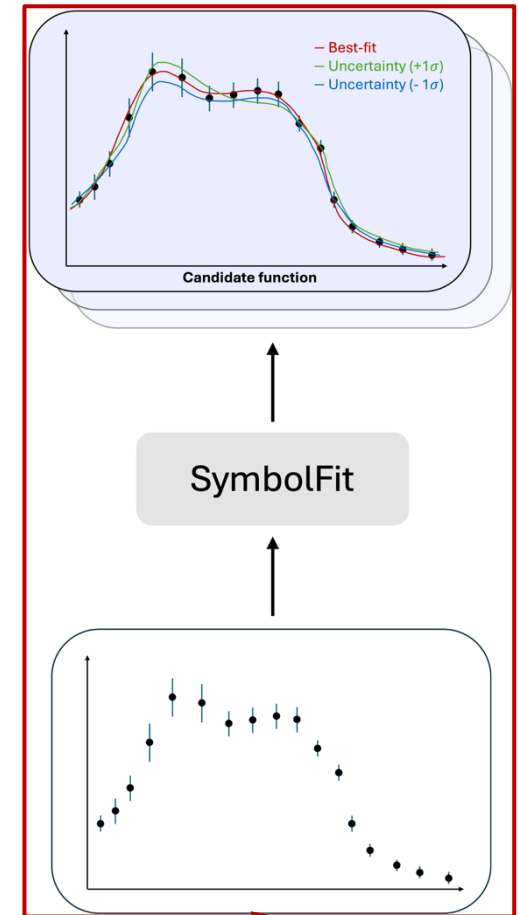
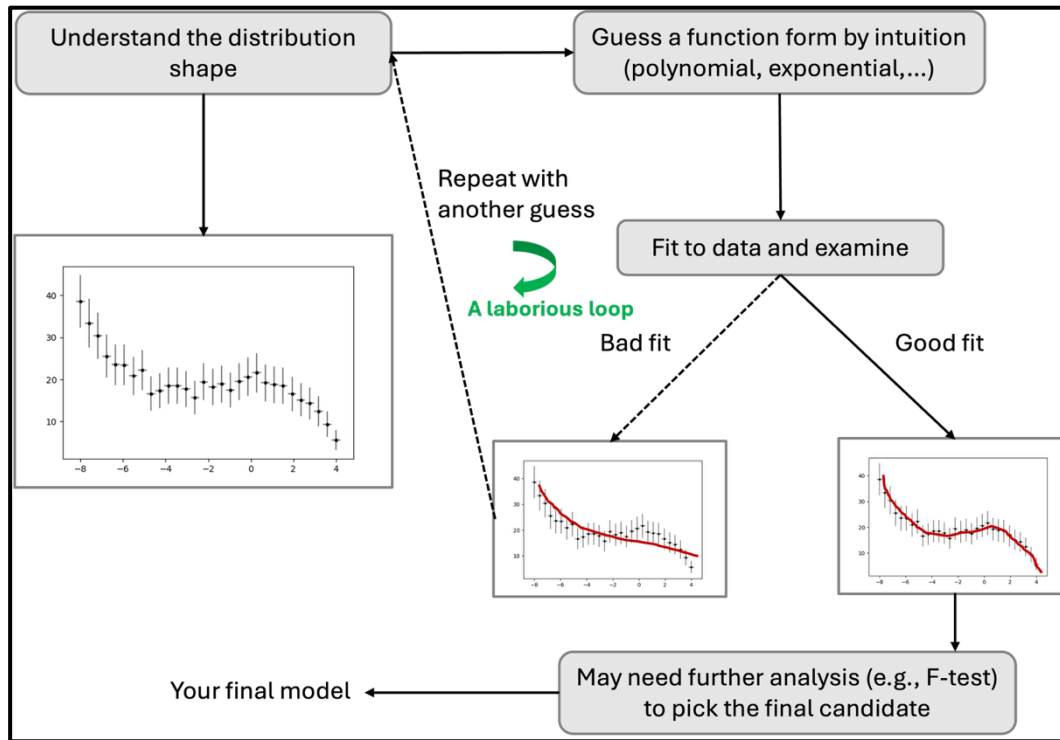
The pain point of the traditional way



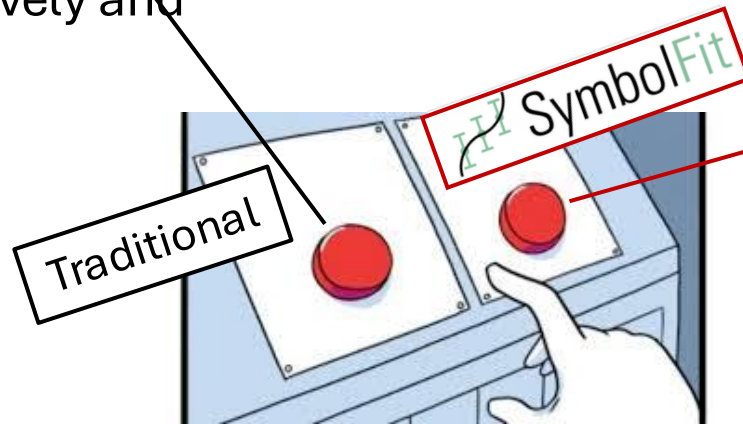
- What about arbitrary shapes?



Now these can be automated by machine

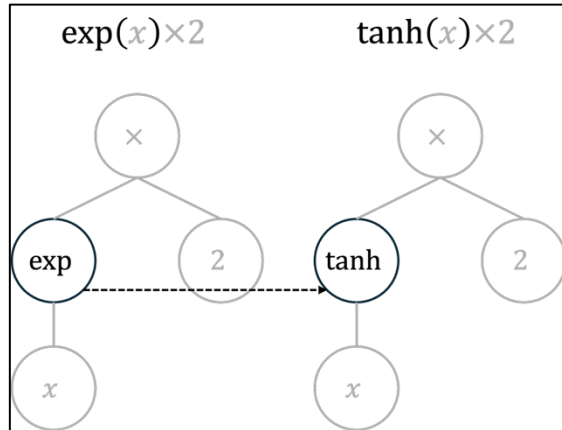


Guess function intuitively and parameterize by hand

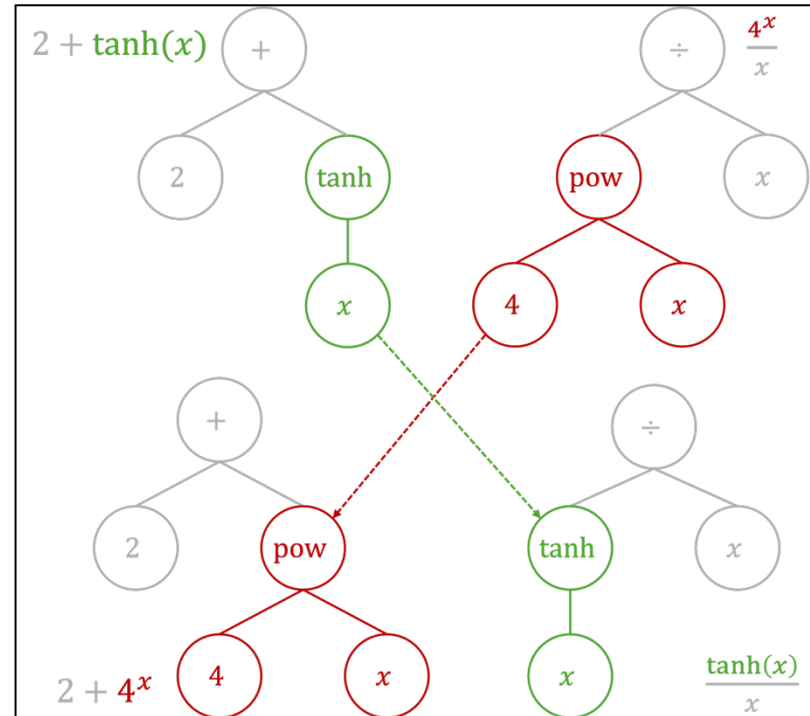


Data-driven search over function space automated by machine

What is symbolic regression?



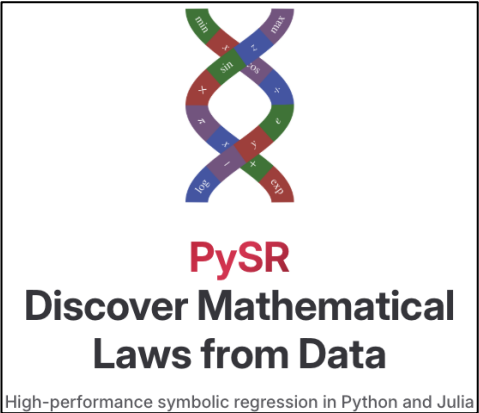
Node mutation



Subtree crossover

- *Machine-search* over function space to find functions to fit data
- Functional form is “trainable”
- You only define what math operators are allowed and the constraints when combining them
- You don't need to know what an adequate function should look like before the fit

PySR and SymbolicRegression.jl

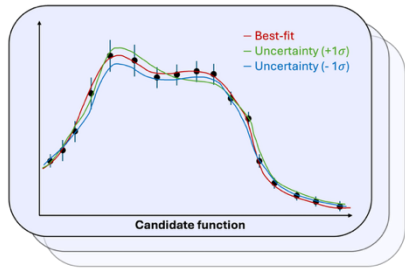


- Source animation from PySR [here](#)
- PySR: high-performance evolutionary SR library in Python [2305.01582](#)

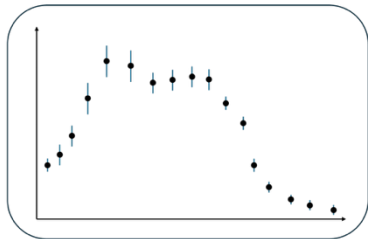
The SymbolFit API

Output

(Multiple candidate functions, each with uncertainty measure)



SymbolFit



Input

(Without a pre-defined functional form)

- Github: [here](#)
- Documentation: [here](#)



[hftsoi/symbolfit](#)

Automatic parametric modeling with symbolic regression

python

machine-learning

automation

interpolation

symbolic-regression

What is SymbolFit?

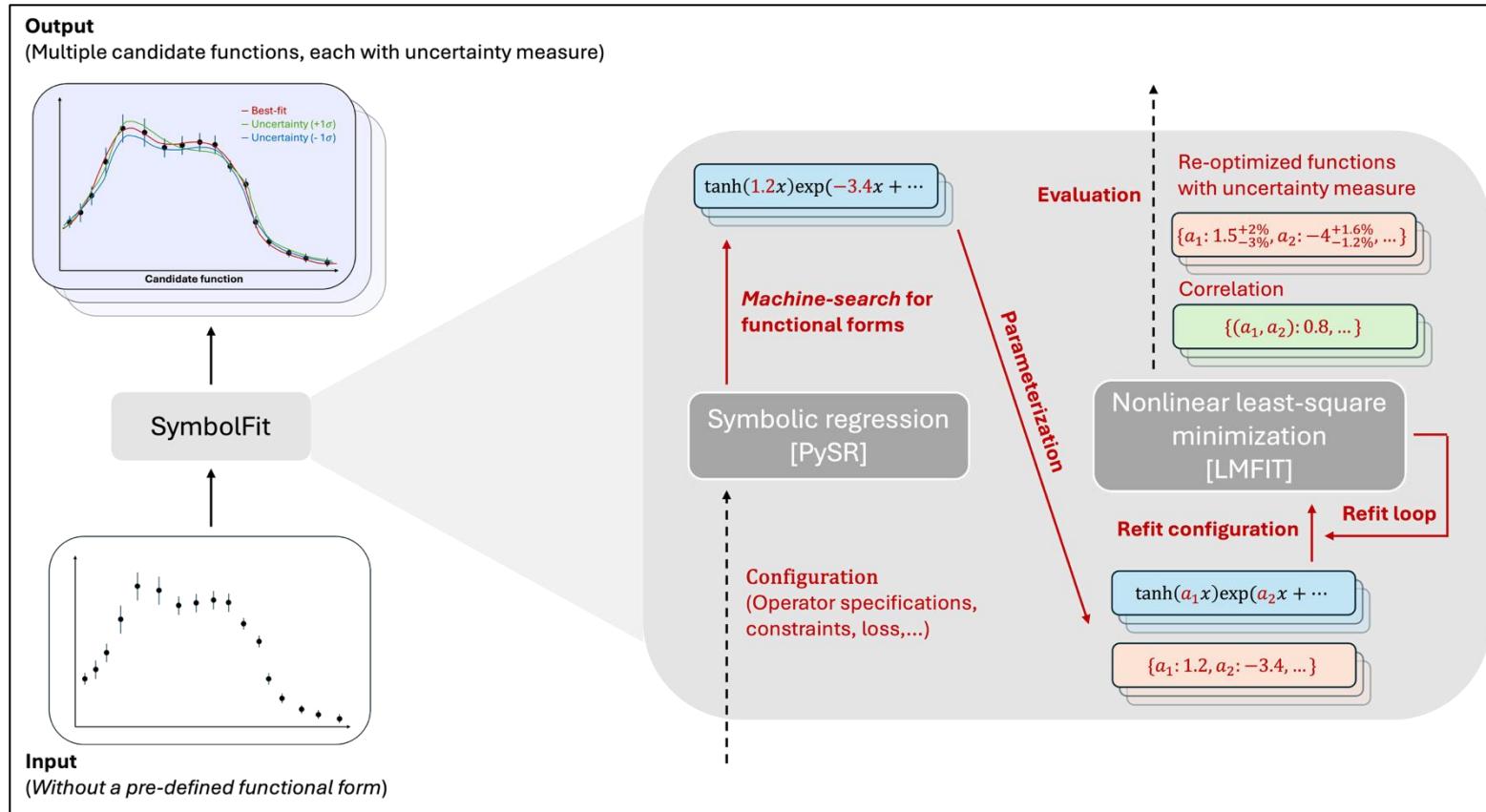
SymbolFit automatically finds closed-form functions that fit your data with uncertainty estimation, no manual guessing required.

SymbolFit was originally developed for experimental high-energy physics (HEP) analyses, but it works on any 1D, 2D, or higher-dimensional dataset where you need an interpretable parametric model with uncertainty estimates. You provide data points and SymbolFit returns a batch of candidate functions ranked by goodness-of-fit, each with optimized parameters and uncertainty estimates. All results are saved to CSV tables and PDF plots, ready for downstream use such as hypothesis testing in HEP.

Under the hood, it chains three steps into a single pipeline:

1. **Function search:** **PySR** (symbolic regression) explores combinations of mathematical operators to discover functional forms that fit the data, without requiring a predefined template.
2. **Re-optimization:** **LMFIT** re-optimizes the numerical parameters in each candidate function and provides uncertainty estimates via covariance matrices.
3. **Evaluation:** every candidate is automatically scored (chi2/NDF, p-value, RMSE, R2), plotted with individual uncertainty variations and total uncertainty coverage, and saved to output files.

Internal steps within SymbolFit



Under the hood, it chains three steps into a single pipeline:

1. **Function search:** **PySR** (symbolic regression) explores combinations of mathematical operators to discover functional forms that fit the data, without requiring a predefined template.
2. **Re-optimization:** **LMFIT** re-optimizes the numerical parameters in each candidate function and provides uncertainty estimates via covariance matrices.
3. **Evaluation:** every candidate is automatically scored (chi2/NDF, p-value, RMSE, R2), plotted with individual uncertainty variations and total uncertainty coverage, and saved to output files.

Configure the function space

```
from pysr import PySRRegressor
import sympy

pysr_config = PySRRegressor(
    model_selection = 'accuracy',
    iterations = 200,
    maxsize = 60,
    binary_operators = [
        '+', '*'
    ],
    unary_operators = [
        'exp',
        'gauss(x) = exp(-x*x)',
        'tanh',
    ],
    nested_constraints = {
        'tanh': {'tanh': 0, 'exp': 0, 'gauss': 0, '*': 2},
        'exp': {'tanh': 0, 'exp': 0, 'gauss': 0, '*': 2},
        'gauss': {'tanh': 0, 'exp': 0, 'gauss': 0, '*': 2},
        '*': {'tanh': 1, 'exp': 1, 'gauss': 1, '*': 2},
    },
    extra_sympy_mappings={
        'gauss': lambda x: sympy.exp(-x*x),
    },
    elementwise_loss='loss(y, y_pred, weights) = (y - y_pred)^2 * weights',
)
```

You only define what math operators are allowed and the rules to connect them

- Allowed math operators: +, -, *, /, ^, exp, tanh,...
- Operator nesting: constraining “exp(exp(exp(x)))” ...
- Equation complexity

See docs [here](#) [symbolfit] and [here](#) [pysr]

You can also specify a form template

```
from pysr import PySRRegressor, TemplateExpressionSpec

expression_spec = TemplateExpressionSpec(
    'p[1] * f(x/13000) ^ g(log(x/13000))',
    expressions = ['f', 'g'],
    variable_names = ['x'],
    parameters = {'p': 1}
)

pysr_config = PySRRegressor(
    expression_spec = expression_spec,
    model_selection = 'accuracy',
    niterations = 200,
    maxsize = 40,
    binary_operators = ['+', '*'],
    elementwise_loss='loss(y, y_pred, weights) = (y - y_pred)^2 * weights',
)
```

If you are confident about a general expression form but not sure about the exact sub-expressions, you can configure by narrowing the function space

- E.g., fixing $f = g(x)^{h(\log(x))}$ and let machine search for g and h
- If “+” and “*” are the only allowed operators, then $g(x)$ will be a polynomial in x and $h(\log(x))$ will be a polynomial in $\log(x)$

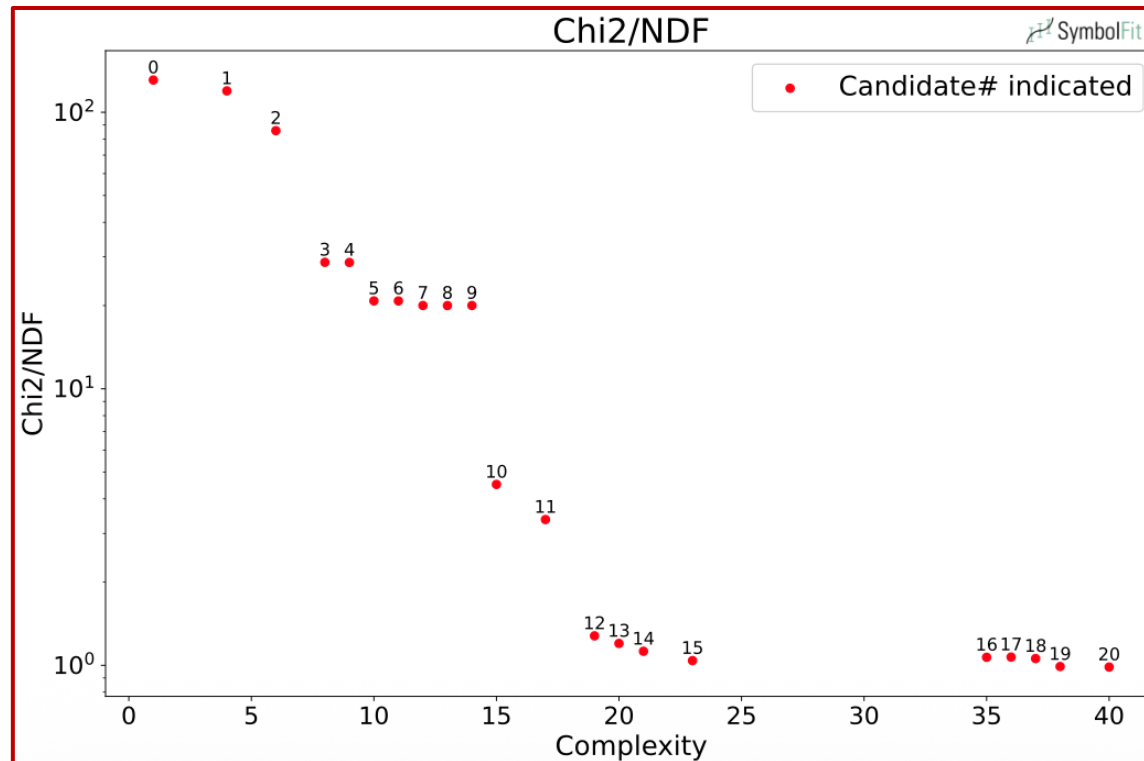
See docs [here](#) [symbolfit] and [here](#) [pysr]

What SymbolFit produces?

File	What it contains
<code>candidates.csv</code>	All candidate functions with parameters, uncertainties, covariance matrices, and goodness-of-fit scores
<code>candidates_compact.csv</code>	Compact version with only final functions, parameters, and key metrics
<code>candidates.pdf</code>	Each candidate plotted against data, with per-parameter uncertainty variations and residual panels
<code>candidates_sampling.pdf</code>	Total uncertainty bands from Monte Carlo parameter sampling (1D only)
<code>candidates_gof.pdf</code>	Summary of goodness-of-fit metrics (chi2/NDF, p-value, RMSE, R2) across all candidates
<code>candidates_correlation.pdf</code>	Parameter correlation matrices for each candidate

What SymbolFit produces?

File	What it contains
<code>candidates.csv</code>	All candidate functions with parameters, uncertainties, covariance matrices, and goodness-of-fit scores
<code>candidates_compact.csv</code>	Compact version with only final functions, parameters, and key metrics
<code>candidates.pdf</code>	Each candidate plotted against data, with per-parameter uncertainty variations and residual panels
<code>candidates_sampling.pdf</code>	Total uncertainty bands from Monte Carlo parameter sampling (1D only)
<code>candidates_gof.pdf</code>	Summary of goodness-of-fit metrics (chi2/NDF, p-value, RMSE, R2) across all candidates
<code>candidates_correlation.pdf</code>	Parameter correlation matrices for each candidate



See output files from an example fit [here](#)

What SymbolFit produces?

File	What it contains
<code>candidates.csv</code>	All candidate functions with parameters, uncertainties, covariance matrices, and goodness-of-fit scores
<code>candidates_compact.csv</code>	Compact version with only final functions, parameters, and key metrics
<code>candidates.pdf</code>	Each candidate plotted against data, with per-parameter uncertainty variations and residual panels
<code>candidates_sampling.pdf</code>	Total uncertainty bands from Monte Carlo parameter sampling (1D only)
<code>candidates_gof.pdf</code>	Summary of goodness-of-fit metrics (chi2/NDF, p-value, RMSE, R2) across all candidates
<code>candidates_correlation.pdf</code>	Parameter correlation matrices for each candidate

	Parameterized equation, unscaled
22	$164.796 \cdot (a_2 + (a_3 \cdot \text{gauss}(a_1 + a_5 \cdot ((x_0 - 12.5) \cdot 0.00210526))) + a_4 \cdot ((x_0 - 12.5) \cdot 0.00210526)) \cdot \text{gauss}(3 \cdot ((x_0 - 12.5) \cdot 0.00210526)))$
23	$164.796 \cdot (a_3 + (a_5 \cdot \text{gauss}((a_1 + ((x_0 - 12.5) \cdot 0.00210526)) \cdot (a_2 + 3 \cdot ((x_0 - 12.5) \cdot 0.00210526)))) + a_6 \cdot ((x_0 - 12.5) \cdot 0.00210526)) \cdot \text{gauss}(a_4 \cdot ((x_0 - 12.5) \cdot 0.00210526)))$

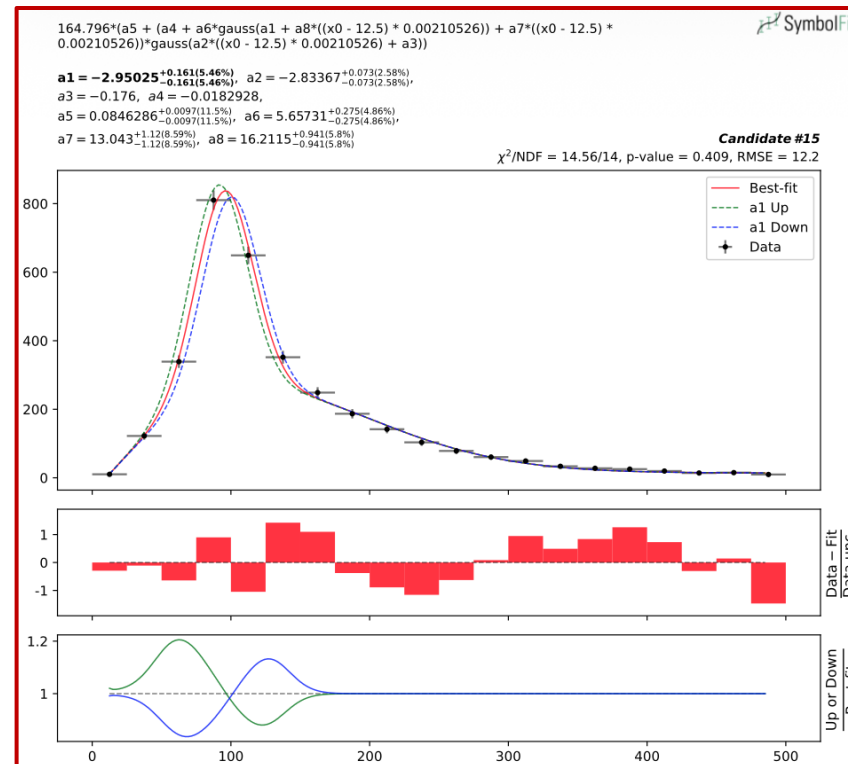
	Parameters: (best-fit, +1, -1)
22	{ 'a1': (-2.78767, 0.123, -0.123), 'a2': (0.0818628, 0.00943, -0.00943), 'a3': (4.81147, 0.24, -0.24), 'a4': (9.97482, 0.438, -0.438), 'a5': (15.4212, 0.743, -0.743)}
23	{ 'a1': (-5.36131, 0.311, -0.311), 'a2': (-0.540169, 0.00724, -0.00724), 'a3': (0.0839784, 0.0104, -0.0104), 'a4': (3.03438, 0.0856, -0.0856), 'a5': (4.80136, 0.244, -0.244), 'a6': (10.3024, 0.992, -0.992)}

	Correlation
22	{ 'a1, a2': 0.02166, 'a1, a3': -0.4448, 'a1, a4': -0.3793, 'a1, a5': -0.9591, 'a2, a3': 0.07339, 'a2, a4': -0.3556, 'a2, a5': -0.03751, 'a3, a4': -0.2259, 'a3, a5': 0.4064, 'a4, a5': 0.4174}
23	{ 'a1, a2': -0.368, 'a1, a3': -0.1906, 'a1, a4': -0.5519, 'a1, a5': -0.3075, 'a1, a6': -0.6573, 'a2, a3': -0.04763, 'a2, a4': 0.02927, 'a2, a5': -0.01356, 'a2, a6': 0.1424, 'a3, a4': 0.4195, 'a3, a6': -0.0001, 'a4, a5': 0.4195, 'a4, a6': 0.0001, 'a5, a6': 0.0001}

	RMSE	R2	NDF	Chi2	Chi2/NDF	p-value
22	13.09	0.9963	15	18.06	1.204	0.2597
23	11.55	0.9971	14	17.04	1.217	0.254

What SymbolFit produces?

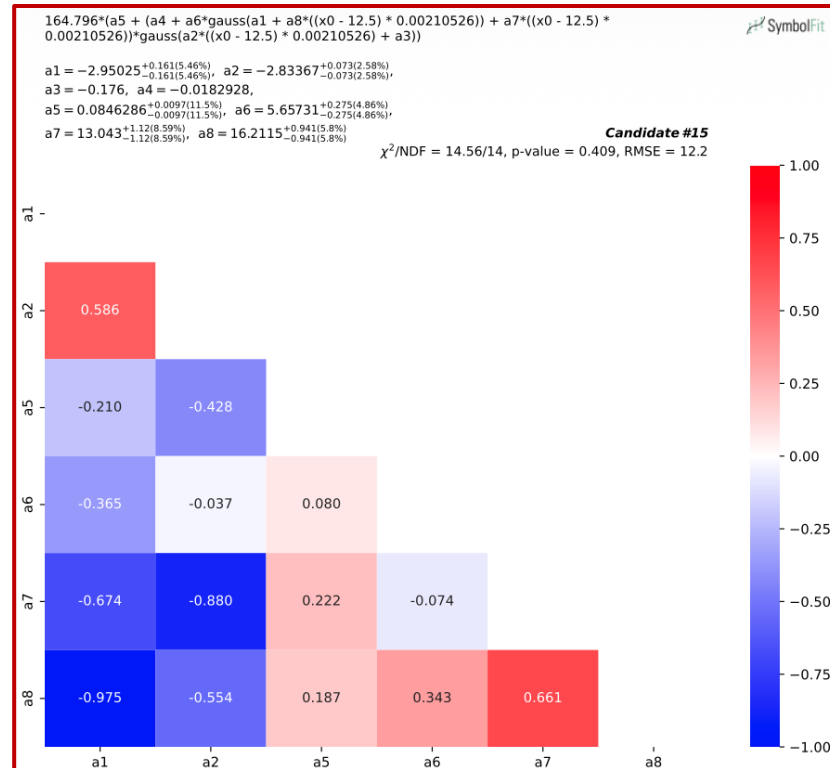
File	What it contains
<code>candidates.csv</code>	All candidate functions with parameters, uncertainties, covariance matrices, and goodness-of-fit scores
<code>candidates_compact.csv</code>	Compact version with only final functions, parameters, and key metrics
<code>candidates.pdf</code>	Each candidate plotted against data, with per-parameter uncertainty variations and residual panels
<code>candidates_sampling.pdf</code>	Total uncertainty bands from Monte Carlo parameter sampling (1D only)
<code>candidates_gof.pdf</code>	Summary of goodness-of-fit metrics (χ^2/NDF , p-value, RMSE, R^2) across all candidates
<code>candidates_correlation.pdf</code>	Parameter correlation matrices for each candidate



See output files from an example fit [here](#)

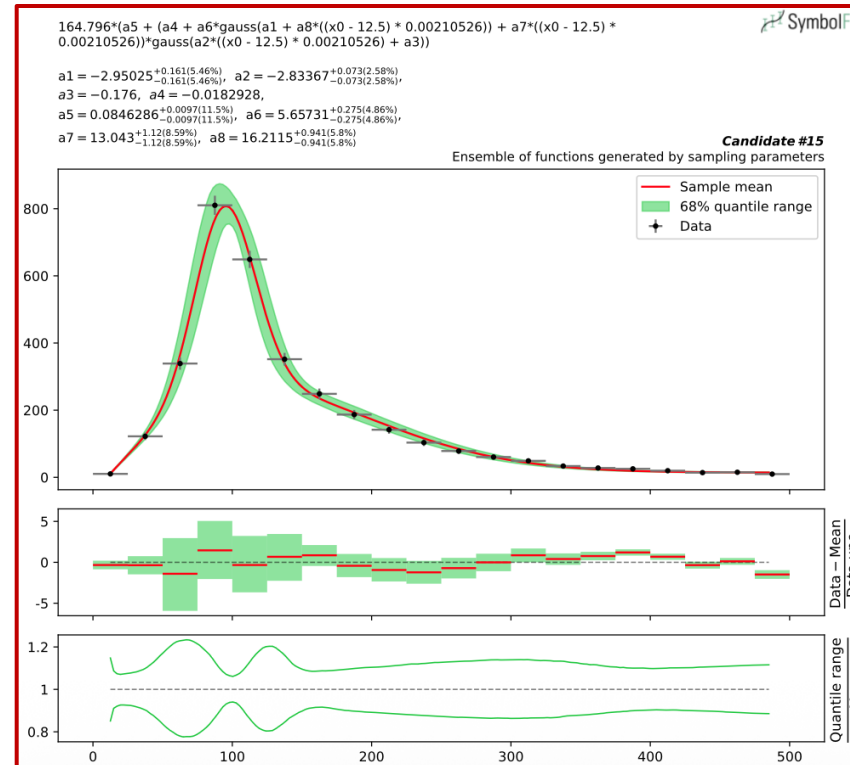
What SymbolFit produces?

File	What it contains
<code>candidates.csv</code>	All candidate functions with parameters, uncertainties, covariance matrices, and goodness-of-fit scores
<code>candidates_compact.csv</code>	Compact version with only final functions, parameters, and key metrics
<code>candidates.pdf</code>	Each candidate plotted against data, with per-parameter uncertainty variations and residual panels
<code>candidates_sampling.pdf</code>	Total uncertainty bands from Monte Carlo parameter sampling (1D only)
<code>candidates_gof.pdf</code>	Summary of goodness-of-fit metrics (chi2/NDF, p-value, RMSE, R2) across all candidates
<code>candidates_correlation.pdf</code>	Parameter correlation matrices for each candidate



What SymbolFit produces?

File	What it contains
<code>candidates.csv</code>	All candidate functions with parameters, uncertainties, covariance matrices, and goodness-of-fit scores
<code>candidates_compact.csv</code>	Compact version with only final functions, parameters, and key metrics
<code>candidates.pdf</code>	Each candidate plotted against data, with per-parameter uncertainty variations and residual panels
<code>candidates_sampling.pdf</code>	Total uncertainty bands from Monte Carlo parameter sampling (1D only)
<code>candidates_gof.pdf</code>	Summary of goodness-of-fit metrics (chi2/NDF, p-value, RMSE, R2) across all candidates
<code>candidates_correlation.pdf</code>	Parameter correlation matrices for each candidate



See output files from an example fit [here](#)

Each run generates tens of candidate functions

164.796*(a1)

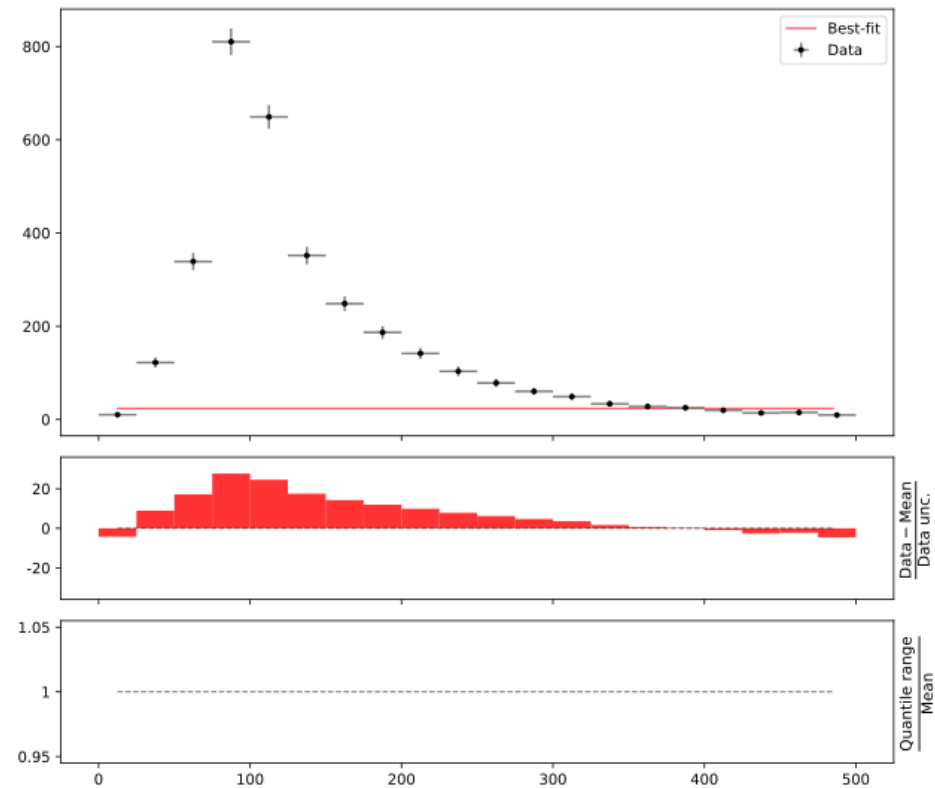
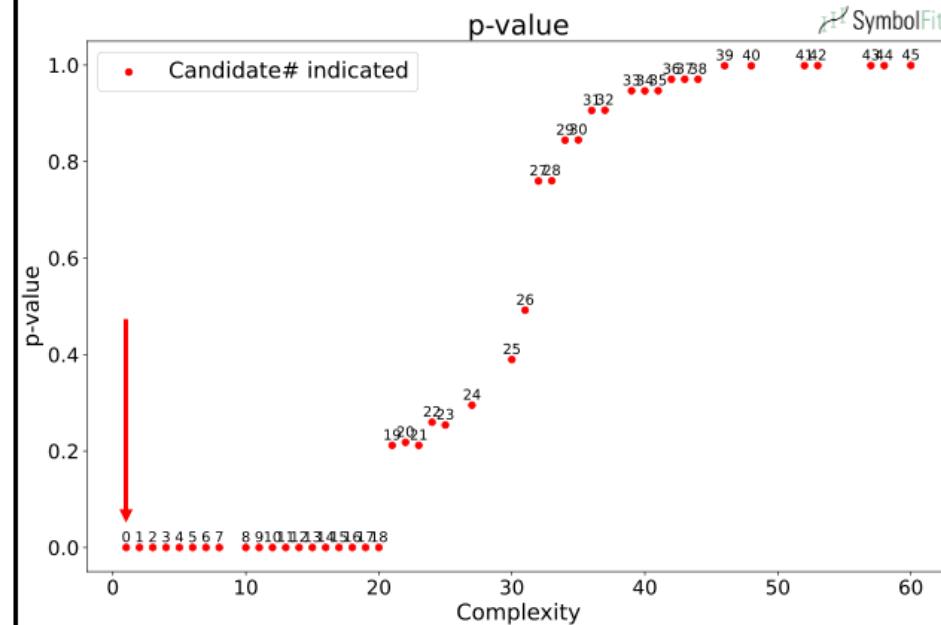
^ Candidate function in parameterized form {a1, a2, a3, ...}

A total of 46 candidate functions were generated from a single fit, each with uncertainty estimates.

SymbolFit

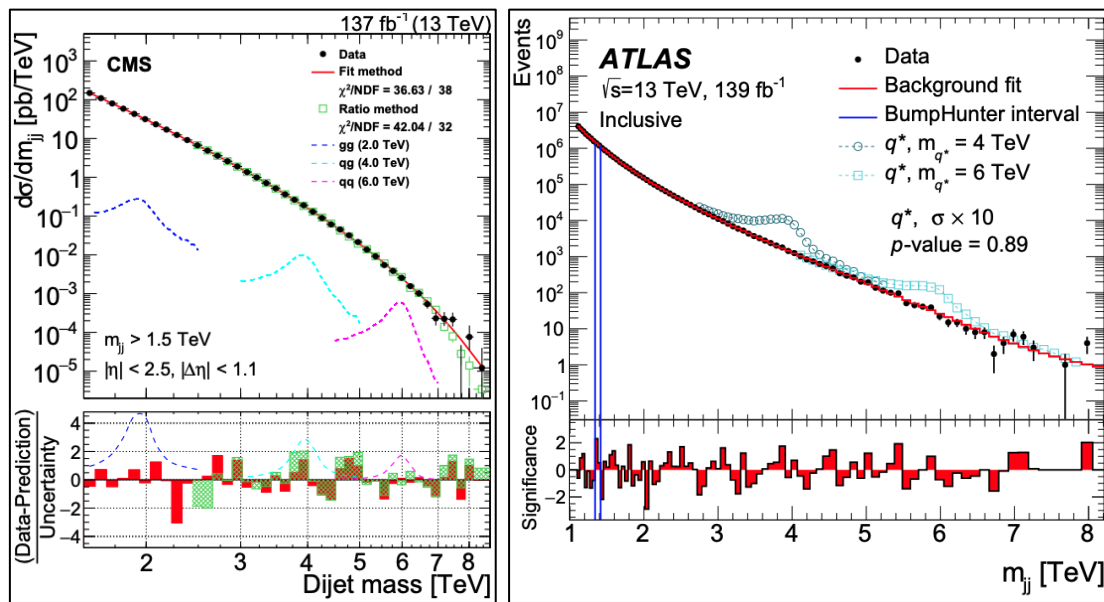
Total uncertainty coverage for each candidate function (68% quantile range obtained by sampling parameters using the covariance matrix).

SymbolFit
Candidate #0



See animation [here](#)

Rediscovering the dijet/UA2 functions from public dijet data



$$f_{\text{dijet}}(x) = \frac{p_0(1-x)^{p_1}}{x^{p_2+p_3 \ln x}}, \quad f_{\text{UA2}}(x) = \frac{p_0 e^{-p_2 x - p_3 x^2}}{x^{p_1}}, \quad x = m_{jj}/\sqrt{s}.$$

We ask:

Given the public Run 2 dijet data from CMS and ATLAS, can we rediscover the very “**dijet functions**” that were empirically invented and used in these published searches?

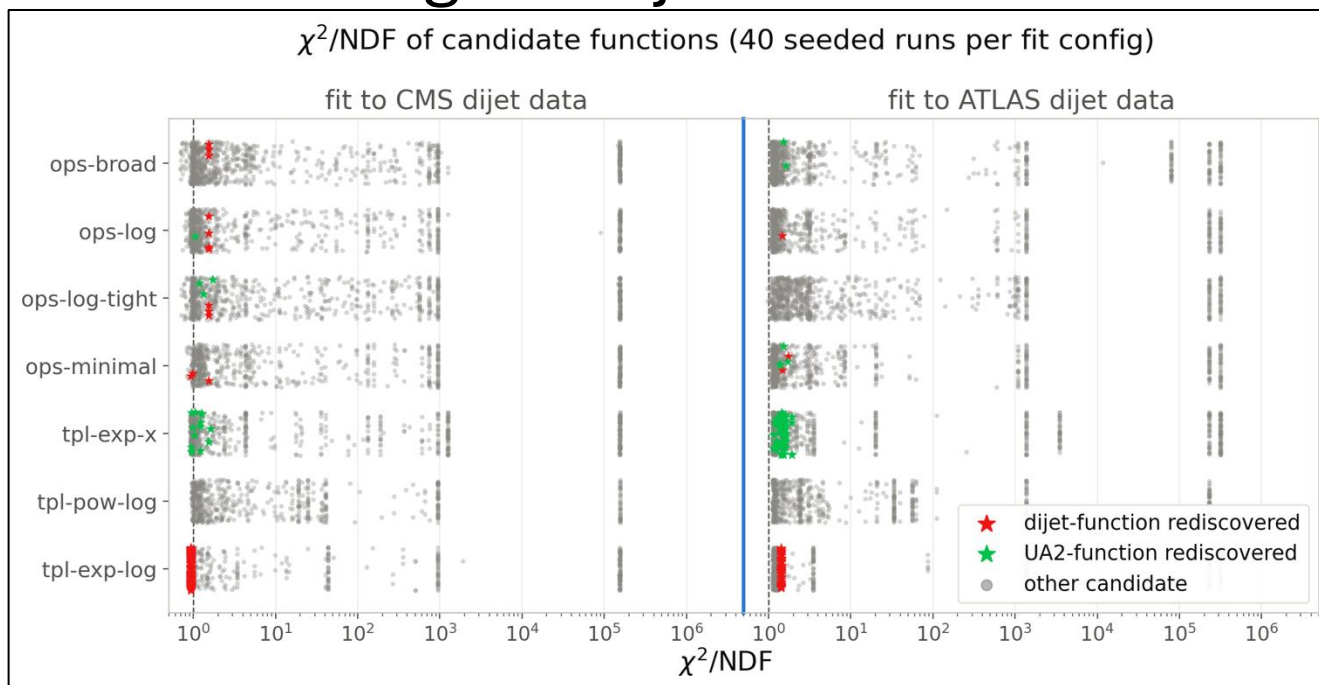
Config variant	Function space	Type
ops-broad	operators $\{+, -, \times, /, \wedge, \exp, \ln, \tanh\}$	free-form
ops-log	operators $\{+, -, \times, /, \wedge, \ln\}$	free-form
ops-log-tight	same as ops-log, with $\wedge$ -subtree sizes limited	free-form
ops-minimal	operators $\{-, \times, /, \wedge\}$	free-form
tpl-exp-x	$f = \exp(h(x))$, with h built from $\{+, -, \times, \ln\}$	template
tpl-pow-log	$f = g(x)^{h(\ln x)}$, with g and h built from $\{+, -, \times\}$	template
tpl-exp-log	$f = \exp(h(\ln x, \ln(1-x)))$, with h built from $\{+, -, \times\}$	template
Common PySR settings: niterations = 200 (free-form) / 100 (template), maxsize = 40, model_selection = accuracy, loss = $(y - \hat{y})^2 / \sigma^2$ per bin		
Common SymbolFit settings: $x = m_{jj}/\sqrt{s}$ fed directly, input_rescale = False, scale_y_by = None, fit_y_unc = True, max_stderr = 20		

Table 1: The seven search configurations. Multiple nesting in operators is generally suppressed. Each config variant is used to perform 40 independent runs with different random seeds on each of the CMS and ATLAS dijet spectra (totaling $7 \times 40 \times 2 = 560$ seeded runs).

We define 7 simple search configurations

- Each fitted 40 times to CMS/ATLAS data with different random seeds
- Total $7 \times 40 \times 2 = 560$ runs

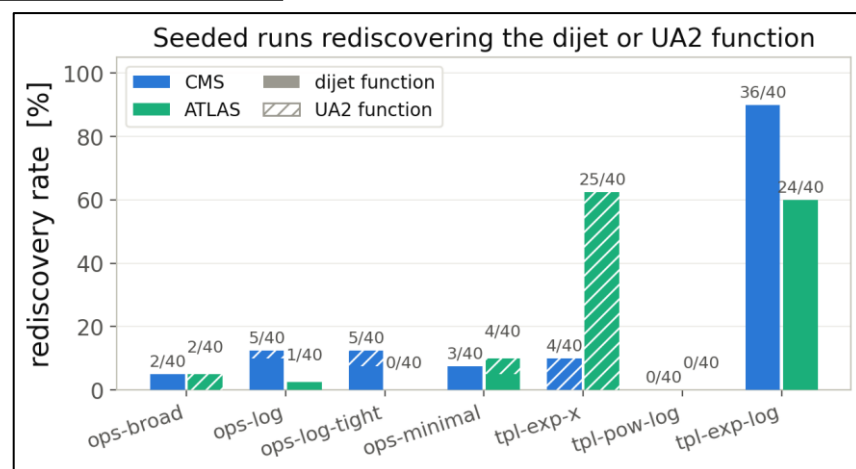
Rediscovering the dijet/UA2 functions from public dijet data



- Non-small rates of rediscovering the dijet/UA2 functions from data
- Tons more candidate functions found to fit as well as the dijet/UA2 functions do

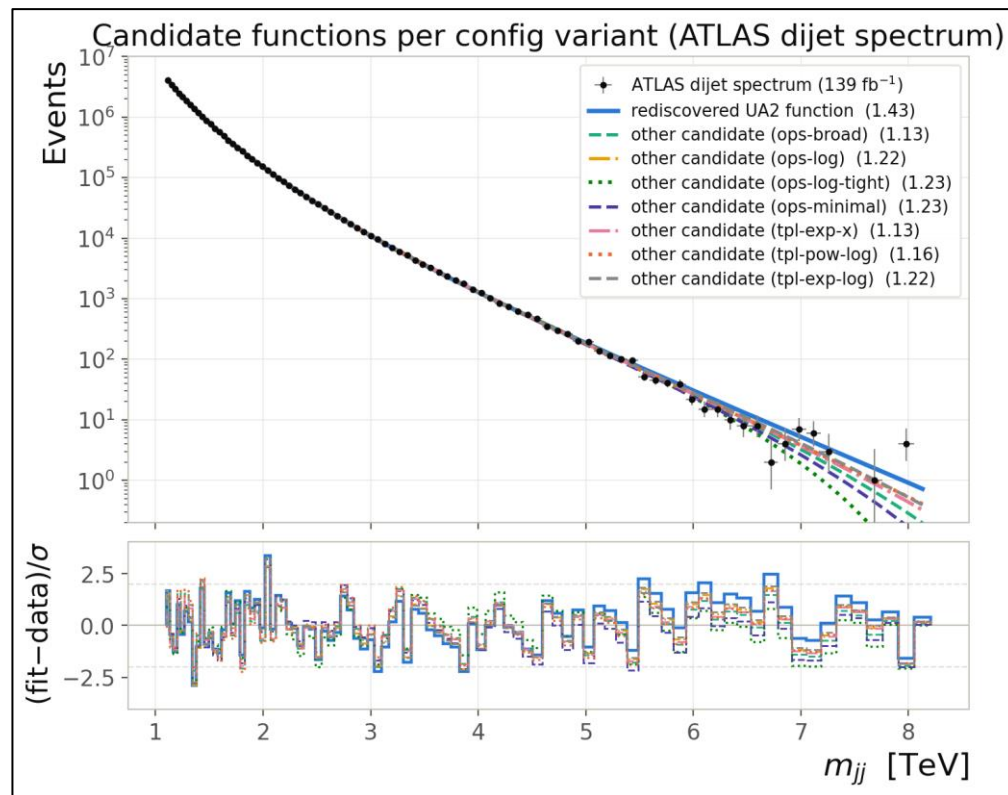
CMS spectrum			ATLAS spectrum		
Config	Candidate function	χ^2/NDF	Candidate function	χ^2/NDF	
rediscovered	$0.00672 (1-x)^{8.15}$	0.93	$0.802 e^{-8.26x-6.12x^2}$	1.43	
	$x^{5.23}$		$x^{5.17}$		
ops-broad	$0.00255 \left(\frac{\ln(0.72/x)}{x} \right)^{4.07}$	1.24	$\left(-1.0 + \frac{0.826}{x + 0.00522} \right)^{5.62}$	1.13	
ops-log	$\left(-\frac{0.176 \ln x}{x} \right)^{x+x^{3.91}+\ln(1.4^x)+4.27}$	0.92	$\left(\frac{0.216x}{x} - 0.315 \right)^{5.11}$	1.22	
ops-log-tight	$(4.05x)^{-7.35x-6.11}$	1.18	$\left(-1.32 + \frac{0.878}{x} \right)^{5.2}$	1.23	
ops-minimal	$\left(\frac{1.66}{x} - 2 \right)^{5.61} \left(1.80 \times 10^{-4} - \frac{4.11 \times 10^{-6}}{x} \right)$	0.97	$\frac{0.71 - x^{0.545/x}}{\left(\frac{0.757 - x}{x} \right)^{4.87}}$	1.23	
tpl-exp-x	$\exp[-(1.40 + \ln x)(1.03 + x)(5.95 + x)]$	1.15	$x + 0.0966$	1.23	
tpl-pow-log	$(0.826 - 0.181x)^{32.2 \ln x + 45.0}$	1.17	$3.07 \exp[x - (2.95x + \ln x) \times (3.94x^2 - x + 4.83)]$	1.13	
tpl-exp-log	$1.01 \exp[(2.63 - 0.0662 \ln(1-x)) \times (3 \ln(1-x) - 2 \ln x - 1.95)]$	0.92	$((x + 1.69)(2.59x^4 + x))^{-0.329 \ln x - 7.04}$	1.16	
			$1.35 \exp[(8.32 - \ln x) \ln(1-x)]$	1.22	
			$x^{5.05}$		

Table 2: The rediscovered functions (dijet for CMS, UA2 for ATLAS) and an example candidate function from each search configuration. Constants are best-fit values (uncertainties are not shown), rounded to three significant figures for readability (χ^2/NDF is computed from full precision). $x = m_{jj}/\sqrt{s}$. The same candidates are plotted in Fig. 3.



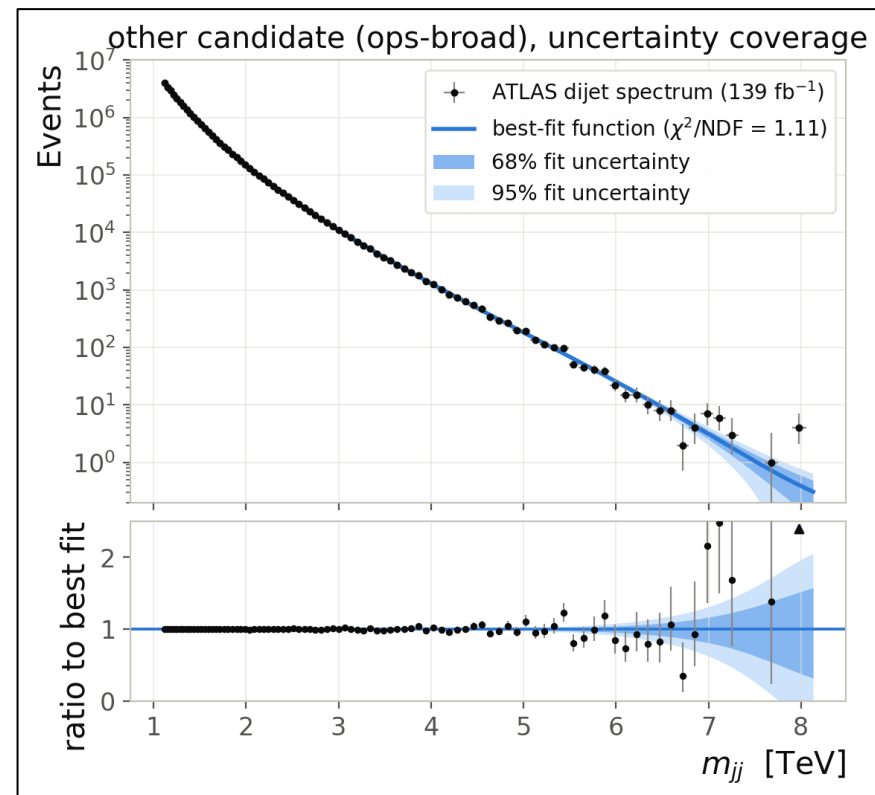
2607.19750

Rediscovering the dijet/UA2 functions from public dijet data



One example candidate function per search config

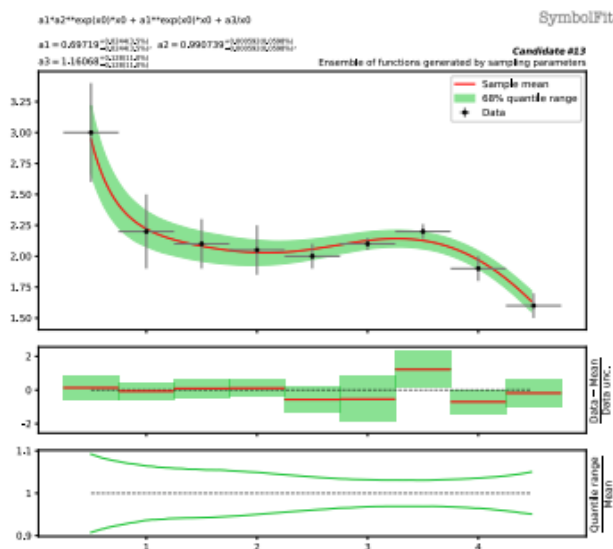
- The “dijet functions” are just a tiny corner
- Thousands of functions found distinct from the “dijet functions” can fit the dijet data equally well



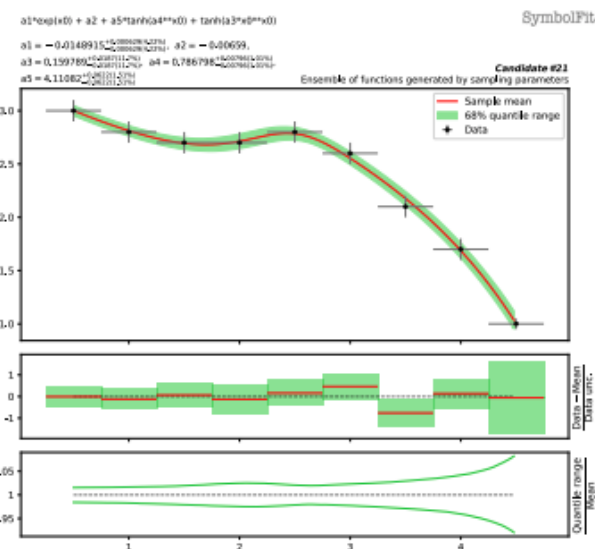
Each candidate function returned has uncertainty estimation

- Individual parameter uncertainty and total uncertainty coverage are automatically plotted and saved to pdf files

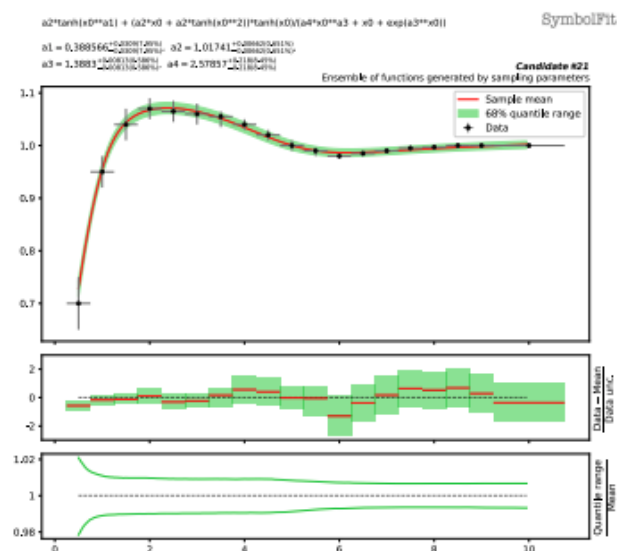
Scale-factor-like examples



(a) Candidate function #13 for Toy Dataset 2a.

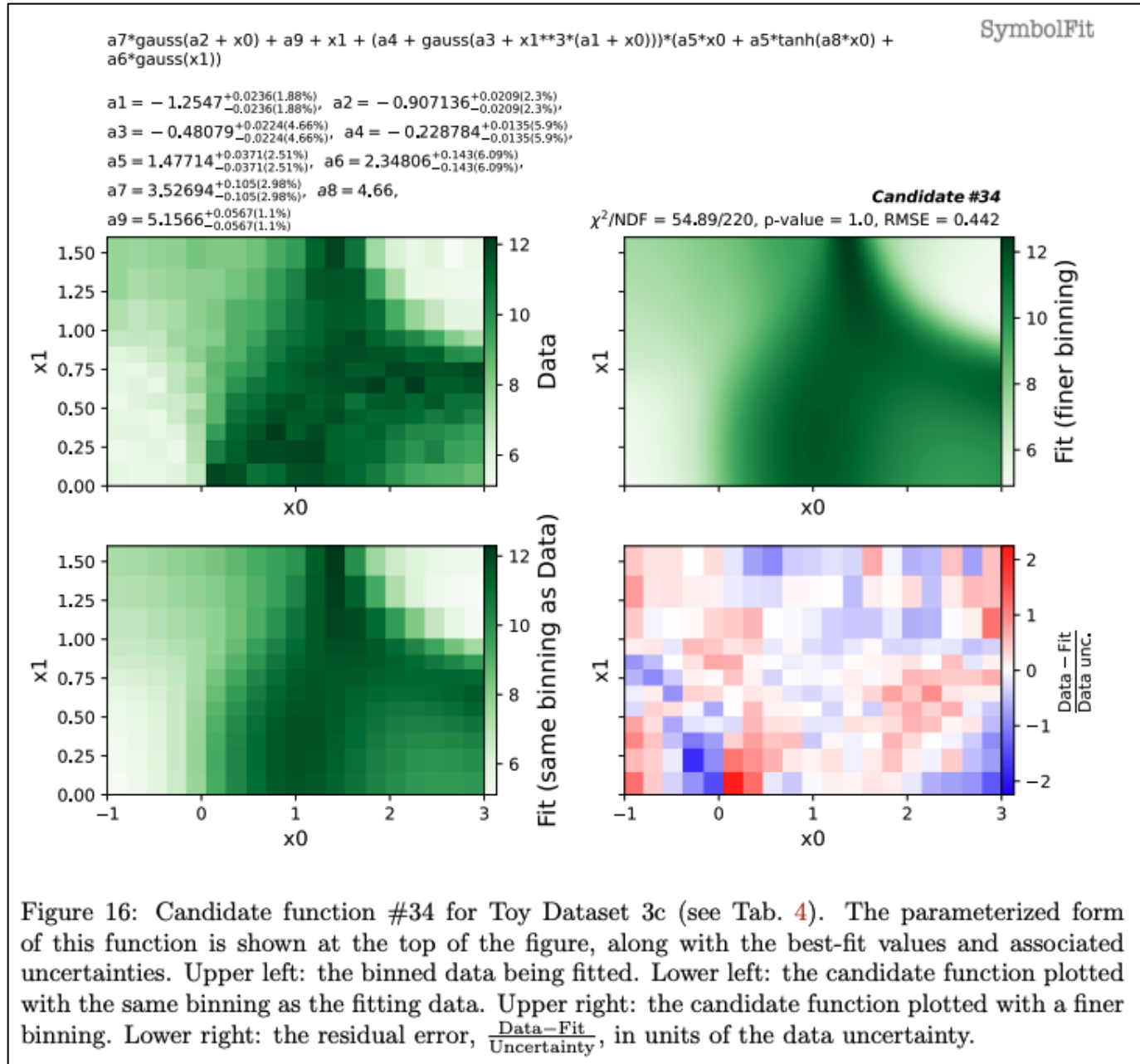


(b) Candidate function #21 for Toy Dataset 2b.



(c) Candidate function #21 for Toy Dataset 2c.

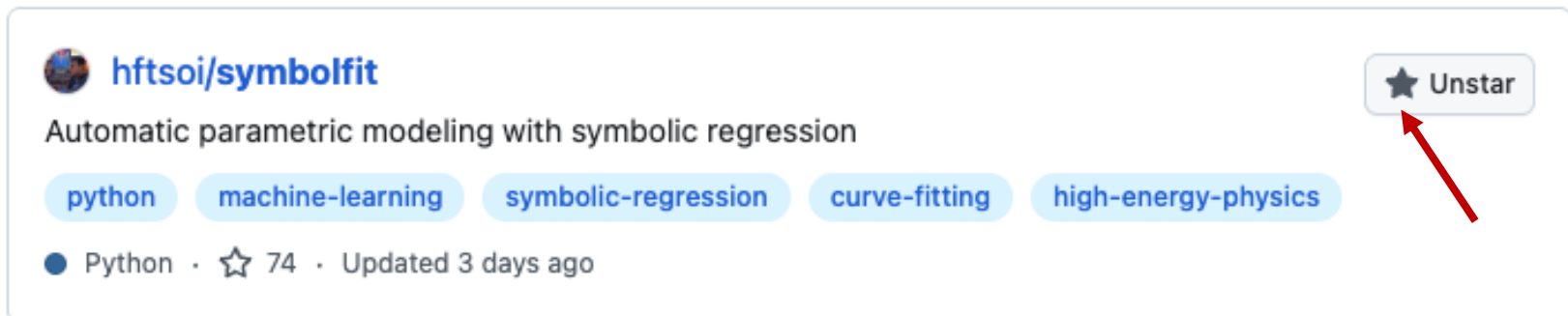
2D data example



Summary

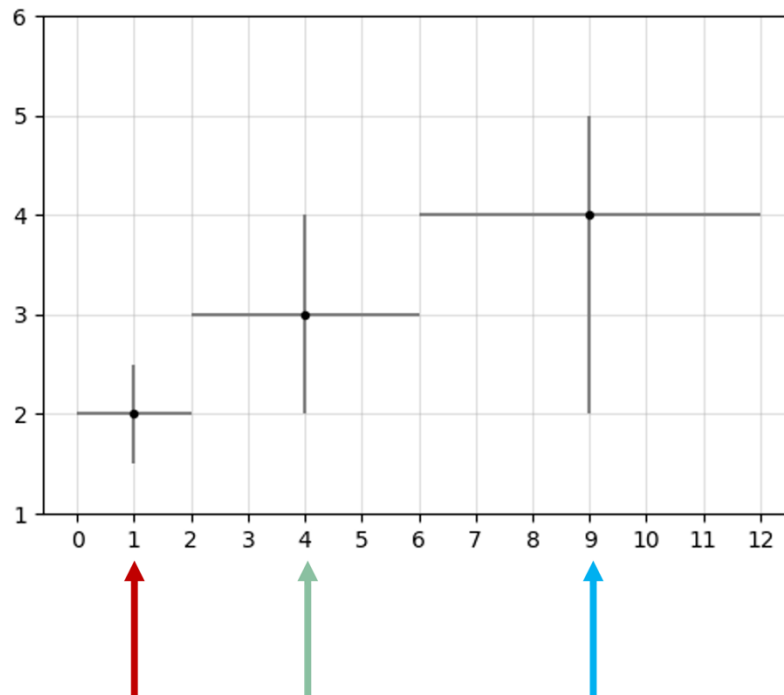
- **SymbolFit** is a python API that interfaces **PySR** [symbolic regression engine] and **LMFIT** [nonlinear optimization and uncertainty estimation]
- It automates parametric modeling for HEP data analysis
- Functions finding, parameterization, uncertainty estimation all automated in one
 - How-to docs: [here](#)
 - Source code: [here](#)
 - Papers: [2411.09851](#) (API) / [2607.19750](#) (dijet function rediscovery)

When in doubt, feel free to drop me a msg and I will be happy to help. Star us if you find this useful 😊



Backup

Input data format (1d)



x = [1, 4, 9]

y = [2, 3, 4]

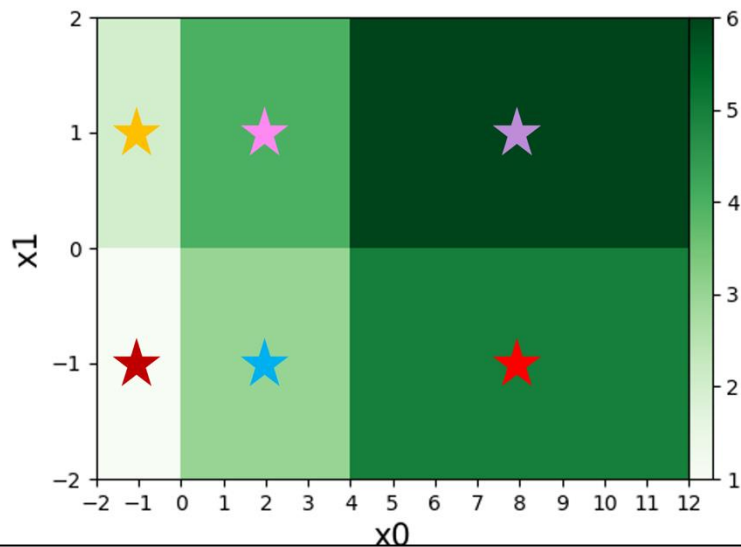
y_up = [0.5, 1, 1]

y_down = [0.5, 1, 2]

bin_widths_1d = [2, 4, 6]

See input format guide [here](#)

Input data format (2d)



```
x = [[-1, -1], [-1, 1], [2, -1], [2, 1], [8, -1], [8, 1]]  
bin_edges_2d = [  
    [-2, 0, 4, 12],  
    [-2, 0, 2]  
]  
y = np.array([1, 2, 3, 4, 5, 6])  
y_up = np.sqrt(y)  
y_down = np.sqrt(y)
```

See input format guide [here](#)

Example: high-mass dijet

SymbolFit easily generated three different functions to model a dijet spectrum comparable to the empirically hand-crafted one by CMS

	Candidate function (after ROF)	# param.	χ^2/NDF (before ROF)	χ^2/NDF (after ROF)	p-value (after ROF)
SR model 1	$(570x(x(-0.423\exp(2x) + \exp(x)) + x) + 149) \times (0.00328 + 0.0304 \tanh(x))^{4.87x}$	5	400.5 / 30	29.21 / 30	0.507
SR model 2	$(145(0.958 + x)^{\tanh(-0.711+4.32x)} + 145 \tanh(x)) \times (0.00591 + 0.0522 \tanh(x))^{5.48x}$	5	103.8 / 30	29.91 / 30	0.47
SR model 3	$\text{pow}(149(0.0101x + 0.0101 \tanh(0.171 + 0.724x)), x + (2.38x \tanh(-0.71 + x) + 2.39) \tanh(x) + \tanh(x))$	5	214.8 / 30	30.93 / 30	0.419

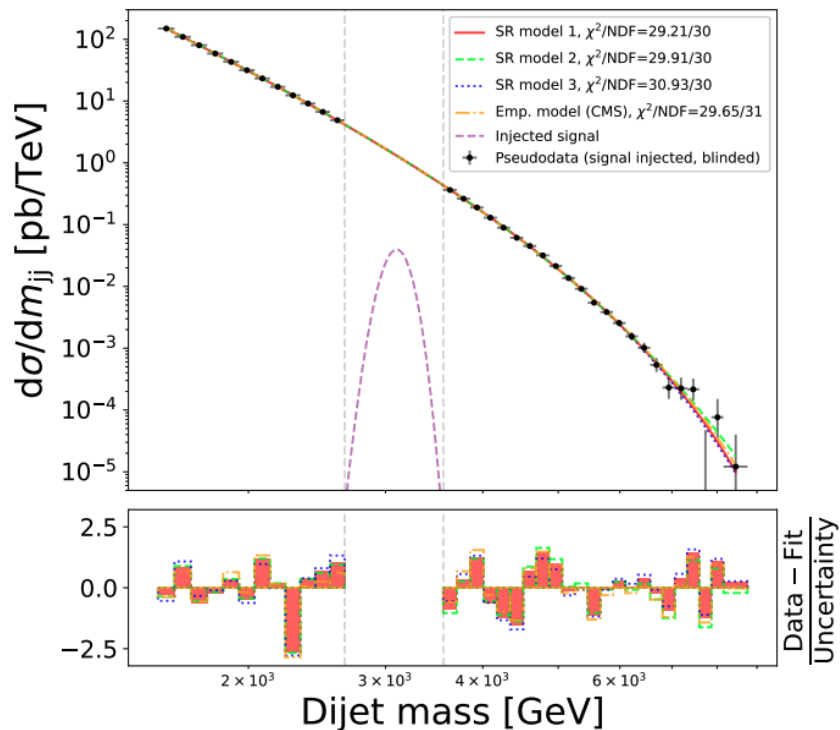


Figure 9: Pseudodata of the dijet spectrum with the injected signal shown in the blinded signal region. The three SR models (see Tab. 2) are compared against the empirical model used by CMS. The lower panel shows the residual error per bin, measured in units of the data uncertainty.

The traditional way

5 Background prediction methods

In the fit method, utilized here and in previous dijet resonance searches [17, 19–32, 50], the main background in the SR coming from QCD is parametrized with an empirical function of the form

$$\frac{d\sigma}{dm_{jj}} = \frac{P_0(1-x)^{P_1}}{x^{P_2+P_3 \ln(x)}}, \quad (1)$$

where $x = m_{jj}/\sqrt{s}$, and P_0, P_1, P_2 , and P_3 are four free parameters. The search for resonances

CMS dijet [1911.03947]

Example: high-mass diphoton

SymbolFit easily generated three different functions to model a diphoton spectrum comparable to the empirically hand-crafted one by CMS

	Candidate function (after ROF)	# param.	χ^2/NDF (before ROF)	χ^2/NDF (after ROF)	p-value (after ROF)
SR model 1	$33.3(0.017 + 0.177x)^{6.93x}$	3	47.12 / 136 = 0.3465	46.83 / 136 = 0.3443	1.0
SR model 2	$35.0 \exp(-16.0x^{0.834})$	3	63.44 / 136 = 0.4665	53.37 / 136 = 0.3925	1.0
SR model 3	$0.0226 \exp(0.975x)(0.158 + x)^{-3.96}$	3	48.94 / 136 = 0.3598	47.22 / 136 = 0.3472	1.0

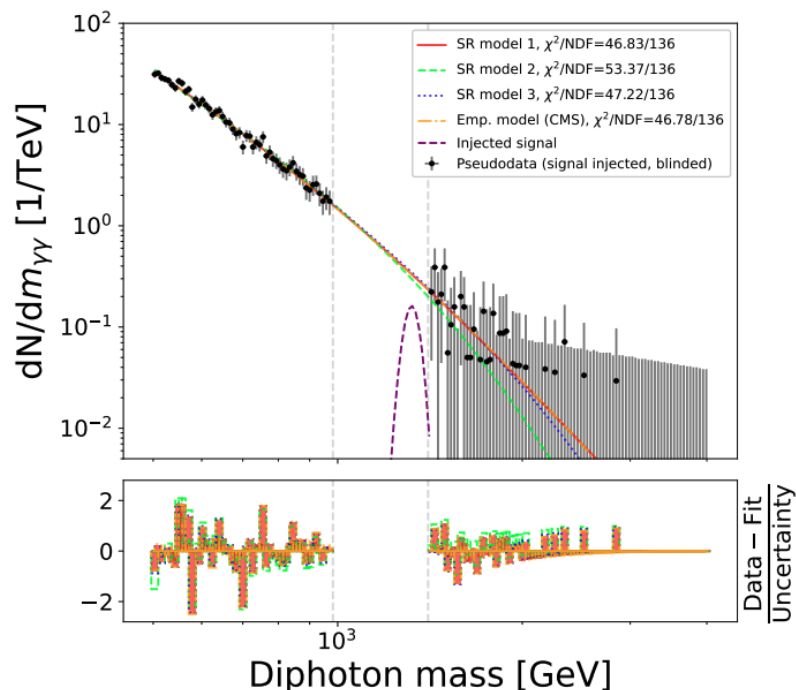


Figure A5: Pseudodata of the diphoton spectrum with the injected signal shown in the blinded signal region. The three SR models (see Tab. A2) are compared against the empirical model used by CMS. The lower panel shows the residual error per bin, measured in units of the data uncertainty.

The traditional way

5.1 Resonant search background

For the resonant signal search, parametrized functions are fit to the $m_{\gamma\gamma}$ spectrum. Localized excesses in the form of a peak are distinguishable from the smoothly-falling background. Four functions of the dimensionless variable $x = m_{\gamma\gamma} / \sqrt{s}$ are given equal consideration in the statistical analysis:

$$\begin{aligned}
 f_1(x) &= p_0 x^{p_1 + p_2 \log(x)}, \\
 f_2(x) &= p_0 e^{p_1 x} x^{p_2}, \\
 f_3(x) &= p_0 (1 + x p_1)^{p_2}, \\
 f_4(x) &= p_0 (1 + x p_1)^{p_2 + p_3 x},
 \end{aligned} \tag{4}$$

where p_i are unconstrained nuisance parameters in the hypothesis test. During the likelihood

CMS diphoton [arxiv:2405.09320](https://arxiv.org/abs/2405.09320)

Example: high-mass trijet

SymbolFit easily generated three different functions to model a trijet spectrum comparable to the empirically hand-crafted one by CMS

	Candidate function (after ROF)	# param.	χ^2/NDF (before ROF)	χ^2/NDF (after ROF)	p-value (after ROF)
SR model 1	$(1.08 \times 10^{-5})^{\tanh(x)} / ((0.165 + x) \times \exp(x^2(-1.96 + 4x))^{\tanh(1.17x^2)})$	3	50.46 / 26 = 1.941	49.04 / 26 = 1.886	0.00408
SR model 2	$\exp(x(-10.8 + x)) / (-0.261x \tanh(x \times (-10.9 + x)) + 0.165 + \tanh(x))$	4	39.49 / 25 = 1.58	33.15 / 25 = 1.326	0.1273
SR model 3	$0.0554^{-0.622+4.03x} (0.568 \tanh(2x) + (0.00302 \exp(x)/(1.92 + x))^x)$	4	38.4 / 25 = 1.536	37.31 / 25 = 1.492	0.05395

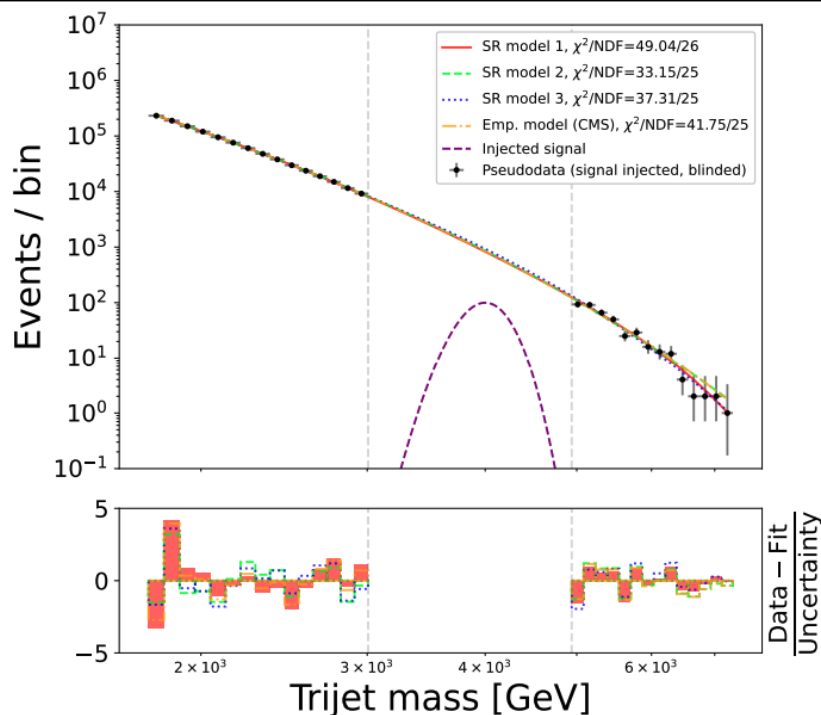


Figure A9: Pseudodata of the trijet spectrum with the injected signal shown in the blinded signal region. The three SR models (see Tab. A4) are compared against the empirical model used by CMS. The lower panel shows the residual error per bin, measured in units of the data uncertainty.

The traditional way

The background, dominantly arising from the QCD multijet process, is modeled by fitting the m_{tj} distribution with smoothly falling, empirical functions that have been used in previous searches [19, 20]. Three families of functions are considered:

$$\begin{aligned}
 f_A(x; N) &= p_0 \frac{(1-x)^{p_1}}{x^{\sum_{i=2}^N p_i \log^{i-2}(x)}}, \\
 f_B(x; N) &= p_0 \frac{e^{-p_1 x}}{x^{\sum_{i=2}^N p_i \log^{i-2}(x)}}, \\
 f_C(x; N) &= p_0 x^{\sum_{i=1}^N p_i \log^{i-1}(x)},
 \end{aligned} \tag{1}$$

where $x = m_{\text{tj}}/\sqrt{s}$, p_i ($i = 0, 1, \dots, N$) are free parameters, and N is the order of the fit function, which is determined using a Fisher F-test [48]. It is found that the third order yields the

CMS trijet [arxiv:2310.14023](https://arxiv.org/abs/2310.14023)

Example: high-mass paired-dijet

SymbolFit easily generated three different functions to model a fourjet spectrum comparable to the empirically hand-crafted one by CMS

	Candidate function (after ROF)	# param.	χ^2/NDF (before ROF)	χ^2/NDF (after ROF)	p-value (after ROF)
SR model 1	$(1.13 \times 10^{-5}x)^{1.28x}/(0.143x^x + 1.63x)$	3	47.95 / 34 = 1.41	39.07 / 34 = 1.149	0.2524
SR model 2	$6.98x^{0.857x}(x + \exp(x))^{-11.8}$	3	47.36 / 34 = 1.393	39.83 / 34 = 1.171	0.2267
SR model 3	$((6.52 \times 10^{-5} + 0.000378x) \tanh(0.641 + 3x))^x / \tanh(0.145 + x)$	3	71.24 / 34 = 2.095	35.57 / 34 = 1.046	0.3942

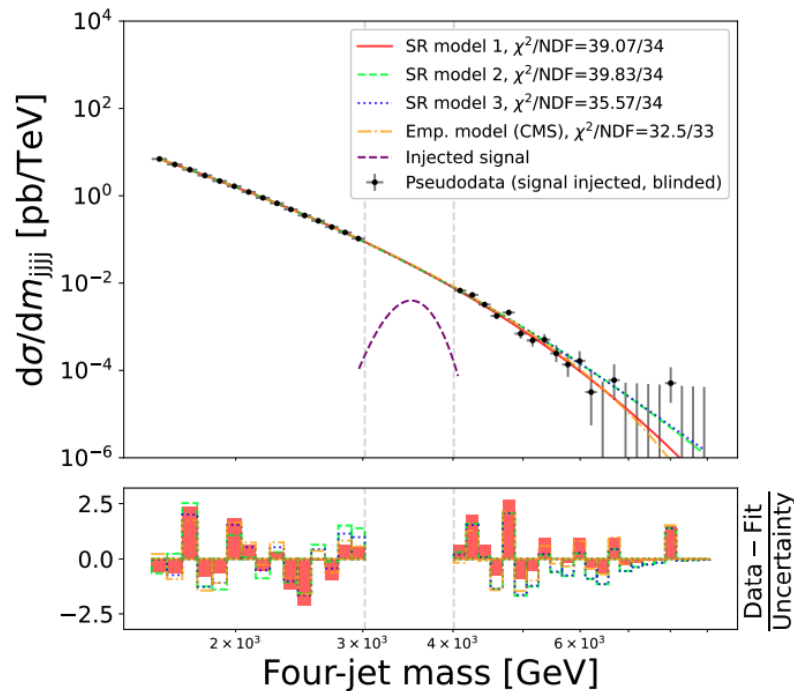


Figure A13: Pseudodata of the four-jet spectrum with the injected signal shown in the blinded signal region. The three SR models (see Tab. A6) are compared against the empirical model used by CMS. The lower panel shows the residual error per bin, measured in units of the data uncertainty.

The traditional way

larger event samples. These are the modified dijet function (ModDijet-5p) given by $d\sigma/dm = p_0(1 - x^{1/3})^{p_1}/(x^{p_2+p_3 \log x + p_4 \log^2 x})$, the dijet function (Dijet-5p) given by $d\sigma/dm = p_0(1 - x)^{p_1}/(x^{p_2+p_3 \log x + p_4 \log^2 x})$, and the power-law times exponential function (PowExp-5p) given by $d\sigma/dm = p_0 e^{-p_1 x - p_2 x^2 - p_3 x^3}/x^{p_4}$, where $x = m/\sqrt{s}$, with m being the four-jet mass.

CMS paired-dijet [arxiv:2206.09997](https://arxiv.org/abs/2206.09997)